

Unidad 1: Ingeniería de software en contexto -----

INTRODUCCIÓN A LA INGENIERÍA DE SOFTWARE

SOFTWARE

El software puede entenderse como un conjunto de programas y la documentación que los acompaña, necesarios para definir, desarrollar, utilizar y mantener el sistema.

Una definición más integradora lo considera conocimiento o información representada en distintos niveles de abstracción: parte de ideas y necesidades expresadas de manera abstracta y se va refinando hasta llegar a representaciones cada vez más concretas, como diseños, código y finalmente información procesable por una computadora (bits 0 y 1)

Por eso, el software no es solamente código.

tres tipos básicos de software. Estos son:

- Software de sistema
- Utilitarios
- Software de Aplicación

Razones para no comparar Software y Manufactura

1. El SW es menos predecible
2. No hay producción en masa, casi ningún producto de SW es igual a otro
3. No todas las fallas son errores
4. El SW no se gasta
5. El SW no está gobernado por las leyes de la física
6. El desarrollo depende fuertemente del factor humano

INGENIERÍA DE SOFTWARE

Ingeniería: Enfoque sistemático y organizado para resolver problemas

Ingeniería de Software: Aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software. Se ocupa de todos los aspectos de su producción, desde la concepción y especificación inicial hasta su operación y mantenimiento.

Saber programar no es suficiente.

Desarrollar software también requiere comprender correctamente el problema, definir requerimientos, diseñar, gestionar el proyecto y las personas, controlar la calidad y mantener el producto.

ANTECEDENTES Y ESTADO ACTUAL

La Ingeniería de Software surge como respuesta a las dificultades que comenzaron a aparecer al desarrollar sistemas cada vez más grandes y complejos.

Algunos hitos importantes en su evolución son:

- 1968: conferencia de la NATO; comienza a utilizarse formalmente el término Software Engineering.
- 1975: The Mythical Man-Month, Frederick Brooks.
- 1987: No Silver Bullet, Brooks.
- 1989: Managing the Software Process, Watts Humphrey.
- 1991: CMM.
- 2001: Manifiesto Ágil.
- 2003: Lean Software Development

Crisis del Software: Refiere al conjunto de dificultades asociadas al desarrollo de software, especialmente evidentes cuando los sistemas comenzaron a crecer en tamaño y complejidad. En la Conferencia de la OTAN de 1968 se planteó la dificultad de producir software libre de defectos, comprensible y verificable.

Causas principales

- Rápida evolución del hardware: permitió desarrollar sistemas mayores y más complejos, pero las prácticas de desarrollo de software no evolucionaron al mismo ritmo.
- Complejidad creciente: se subestimó la dificultad propia de construir software, una actividad humana cuyo producto es abstracto e intangible.
- Mayor demanda de sistemas complejos.
- Cambios constantes para adaptar los productos a las necesidades del cliente.
- Falta de una disciplina sistemática que abordara todos los aspectos de la producción de software.

Estado actual: Aunque las técnicas y herramientas han evolucionado, continúan apareciendo problemas como:

- productos que no satisfacen las necesidades del cliente
- dificultad para extenderlos o adaptarlos. Agregar más funciones en otra versión es casi imposible
- mala documentación
- baja calidad
- proyectos que superan los tiempos y costos presupuestados

factores que mantienen vigente esta problemática:

- Demandas crecientes: se requieren sistemas cada vez más grandes y complejos y se espera que sean desarrollados y entregados rápidamente.
- Bajas expectativas: algunas organizaciones se conforman con que el software simplemente funcione, aunque sea más costoso, menos confiable o aporte poco valor al negocio.

Ejemplos de grandes proyectos de SW fallidos y exitosos

Proyectos / sistemas fallidos:

- Mariner 1 (1962): sonda espacial destinada a Venus que se desvió de su trayectoria poco después del lanzamiento y tuvo que ser destruida. El problema estuvo relacionado con una expresión matemática programada incorrectamente en el sistema de guiado
- Ariane 5 (1996): el cohete explotó pocos segundos después del lanzamiento debido a un error en la conversión de datos numéricos dentro del software de navegación. Un valor que no podía representarse correctamente provocó una falla del sistema de control.
- Mars Climate Orbiter (1999): la NASA perdió la sonda al llegar a Marte porque distintos componentes utilizaban unidades de medida incompatibles: uno trabajaba con unidades inglesas y otro con unidades métricas. La falta de consistencia entre los sistemas produjo cálculos incorrectos de navegación y la pérdida de la misión.

Proyectos exitosos:

- Proyecto Apollo: desarrolló software crítico y altamente confiable para las misiones espaciales. Margaret Hamilton lideró el equipo responsable del software de vuelo, utilizado por el programa Apollo.
- GNU/Linux: combinación del kernel Linux con herramientas del proyecto GNU que se convirtió en un sistema operativo ampliamente utilizado, especialmente en servidores y otras infraestructuras informáticas.
- Apache: servidor web de código abierto que alcanzó una adopción masiva y se convirtió durante años en una tecnología fundamental para servir sitios web.

DISCIPLINAS DE LA INGENIERÍA DE SW

1. DISCIPLINAS TÉCNICAS: Permiten obtener como resultado el producto de software.

- Prueba
- Despliegue
- También incluye: Requerimientos, análisis y diseño, construcción

2. DISCIPLINAS DE GESTIÓN: Enfocadas en el proyecto.

Proyecto: Medio que permite obtener como resultado un producto de software. Es una unidad de gestión de recursos (organización) y personas (trabajo).

Lo producido debe tener resultado únicos y elaboración gradual.

Cada proyecto ejecutado es una versión distinta del producto.

- Planificación de proyecto
- Monitoreo y control de proyecto

3. DISCIPLINAS DE SOPORTE: Acompañan y sostienen todo el proceso para que el resultado sea confiable y controlado

- Gestión de configuración de software: Mantener la integridad del producto de software
- Aseguramiento de calidad: Actividades con técnicas y herramientas que acompañan la creación del producto. No es el testing, trabajar con calidad es durante todo el proyecto, la calidad está relacionada con las expectativas (no se manifiestan) y necesidades (si se manifiestan) del producto, es subjetiva
- Métricas: Es un número que proporciona una valoración objetiva

PAPER NO SILVER BULLETS

La idea central de No Silver Bullet es que no existe una única tecnología, herramienta o técnica capaz de eliminar por sí sola las dificultades fundamentales del desarrollo de software.

Brooks distingue entre dificultades esenciales y accidentales, Esencia vs. accidente

1. Dificultades accidentales: Son dificultades asociadas a cómo representamos y construimos el software, pero no a la naturaleza misma del problema.

Gran parte de la historia de la informática permitió reducirlas mediante:

- lenguajes de programación de alto nivel;
- time-sharing;
- entornos y herramientas de desarrollo;
- mayor capacidad de hardware;
- automatización.

Estas mejoras permiten trabajar más rápido o con menos errores, pero no eliminan la dificultad conceptual de decidir y diseñar qué sistema construir.

2. Dificultades esenciales: Son inherentes a la naturaleza del software y no pueden eliminarse simplemente mediante mejores herramientas. Brooks identifica cuatro:

1. **Complejidad:** los sistemas poseen una enorme cantidad de elementos e interrelaciones. Al crecer el software aumenta fuertemente la dificultad de comprenderlo, diseñarlo, comunicarlo y modificarlo.
2. **Conformidad:** el software debe adaptarse a reglas, sistemas, necesidades y restricciones externas que muchas veces son arbitrarias.
3. **Mutabilidad:** el software está sometido constantemente a cambios porque cambian el negocio, los usuarios, el entorno y los sistemas con los que interactúa.
4. **Invisibilidad:** el software no posee una representación física o geométrica natural que permita visualizar fácilmente toda su estructura.

El verdadero cuello de botella

La parte más difícil no es necesariamente escribir el código, sino decidir exactamente qué debe construirse, especificar correctamente el problema y diseñar su estructura conceptual.

Ejemplo con la IA: una herramienta puede acelerar la generación de código, pero alguien todavía debe expresar correctamente la intención, las restricciones, los estados y la lógica del negocio.

Las herramientas pueden acelerar el cómo, pero no eliminan la dificultad de decidir correctamente el qué.

¿Cómo atacar la esencia?

Propone mejorar el trabajo conceptual mediante distintas estrategias:

- Buy vs. Build: antes de construir desde 0, analizar si existe una solución que pueda comprarse o reutilizarse.
- Rapid Prototyping: construir prototipos para obtener feedback temprano y refinar los requerimientos.
- Grow, not build: hacer crecer el sistema incrementalmente, manteniendo siempre una versión funcional.
- Grandes diseñadores: reconocer que la calidad conceptual depende fuertemente de la capacidad humana para diseñar buenas soluciones.

Podemos reducir muchas dificultades accidentales, pero la complejidad esencial del software permanece. El diseño sigue siendo fundamentalmente una actividad intelectual humana.

PROCESO DE SOFTWARE

Proceso: Secuencia de pasos ejecutados con un propósito determinado.

Proceso de software: Conjunto estructurado de actividades, métodos, prácticas y transformaciones utilizadas por las personas para desarrollar o mantener software y sus productos asociados. Las actividades pueden variar según la organización y el tipo de sistema.

Es una definición abstracta (plantilla) que se vuelve concreta cuando se ejecuta un proyecto

Para que un proceso pueda administrarse, debe estar explícitamente modelado.

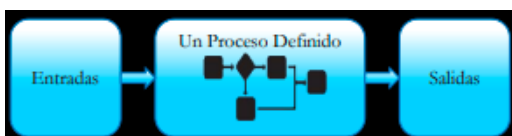
Elementos fundamentales, un proceso combina:

- Procedimientos y métodos: qué actividades se realizan y cómo.
- Personas: habilidades, capacitación, experiencia y motivación.
- Herramientas y equipos: permiten facilitar o automatizar las actividades.

No existe un proceso ideal universal: debe adaptarse al contexto, las personas y las características del producto

PROCESOS DEFINIDOS: Parten de la idea de que el proceso puede especificarse con suficiente detalle y repetirse obteniendo resultados previsibles.

- Están inspirados en las líneas de producción.
- Buscan repetibilidad y predictibilidad.
- La administración y el control se basan en la predictibilidad del proceso.
- La experiencia previa se considera extrapolable, es decir, puede utilizarse para predecir cómo se comportará un nuevo proyecto.

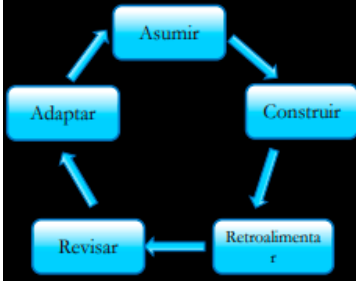


El proceso está preestablecido y especificado de antemano: se conocen las actividades y el orden en que deben ejecutarse, y se espera que ante entradas similares el proceso produzca salidas previsibles.

PROCESOS EMPÍRICOS: Se utilizan en contextos complejos y con variables cambiantes, donde repetir el mismo proceso puede producir resultados diferentes.

- El conocimiento se obtiene a partir de la experiencia.
- La administración y control se realizan mediante inspecciones frecuentes y adaptación.
- Son apropiados para trabajos creativos y complejos.
- La experiencia obtenida depende del contexto y no es directamente extrapolable.

Patrón de conocimiento



El conocimiento proviene de:

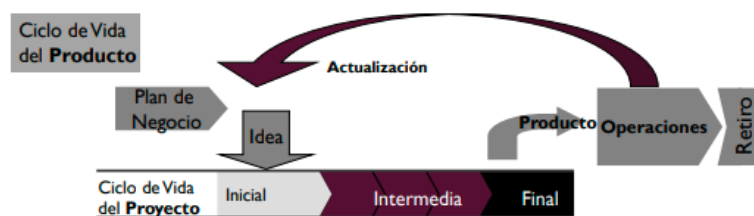


la experiencia individual
las habilidades
las prácticas reales del equipo
el aprendizaje compartido

Se apoyan en Transparencia -> Inspección -> Adaptación

CICLOS DE VIDA

Es la serie de pasos a través de los cuales progresa un producto o proyecto.



El ciclo de vida del producto siempre es mayor que el ciclo de vida del proyecto, ya que el ciclo de vida del proyecto dura lo que dura el desarrollo del software.

En cambio, el ciclo de vida del producto dura hasta que el software se deje de utilizar, dependiendo esto de la madurez que tenga el producto en base a las necesidades del mercado. Es posible que un producto tenga varios proyectos en su ciclo de vida, debido a constantes cambios y/o actualizaciones que se vayan realizando.

Por lo que, dentro del ciclo de vida del producto, se pueden desarrollar varios ciclos de vida de proyectos.

Los ciclos de vida de proyectos de software influyen en la administración de proyectos

Un **ciclo de vida de software** es una representación del proceso desde una determinada perspectiva. Define:

- las fases
- el orden en que se realizan
- la relación entre ellas
- los criterios para avanzar de una fase a otra

Relación entre proceso de desarrollo de software y ciclo de vida (modelos de proceso)

- El proceso de desarrollo establece qué actividades, métodos y prácticas se utilizan para desarrollar software.
- El ciclo de vida determina cómo se organizan y en qué orden o momento se realizan esas actividades.

Un mismo proceso puede aplicarse mediante distintos ciclos de vida, según estrategia y características del proyecto.

Clasificación de los ciclos de vida, 3 tipos básicos

1. Secuencial: Las actividades se realizan en una secuencia lineal de fases. Una etapa comienza cuando la anterior ha finalizado y normalmente existe una revisión antes de avanzar.

Adecuado cuando los requerimientos son claros, completos y estables, existe baja incertidumbre y no se necesitan entregas tempranas

Ventajas:

- ✓ facilita planificar anticipadamente
- ✓ estructura claramente el proyecto
- ✓ facilita utilizar un esquema de gestión uniforme
- ✓ la documentación permite observar el avance.

Desventajas:

- ✗ poca flexibilidad frente a cambios
- ✗ necesita requerimientos bien definidos desde el inicio
- ✗ el cliente recibe software funcionando muy tarde, el feedback del mismo llega tarde
- ✗ se incrementa el riesgo de no satisfacer necesidades reales
- ✗ un defecto descubierto tarde puede provocar gran retrabajo, costo de corregir errores aumenta
- ✗ genera dependencia entre fases/equipos
- ✗ puede producir documentación excesiva

2. Iterativo/Incremental: El sistema se construye mediante iteraciones sucesivas, y cada una produce un incremento funcional potencialmente entregable. Las primeras iteraciones suelen priorizar las funcionalidades más importantes y las actividades de especificación, desarrollo y validación están más entrelazadas.

Es el ciclo de vida que mejor se adapta a los procesos empíricos y Agile.

Ventajas:

- ✓ permite responder mejor a requerimientos cambiantes
- ✓ obtiene feedback frecuente del cliente
- ✓ reduce el retrabajo
- ✓ permite entregar valor antes, hace visible software funcionando tempranamente

Desventajas:

- ✗ administrativamente puede ser más difícil visualizar todo el proceso
- ✗ documentar cada incremento puede resultar costoso
- ✗ si no se cuida la arquitectura, ésta puede degradarse con sucesivos cambios

Iteración en proceso definido vs. empírico: que un ciclo de vida sea iterativo no implica necesariamente que el proceso sea empírico.

- En un **proceso definido**, cada iteración se ejecuta siguiendo actividades y un plan establecidos previamente, buscando repetibilidad y predictibilidad.
- En un **proceso empírico**, la iteración funciona además como ciclo de aprendizaje: se obtiene un resultado, se inspecciona lo ocurrido y se adapta el producto, el plan o la forma de trabajo según la evidencia obtenida.

Por lo tanto, en el definido la iteración busca principalmente ejecutar lo planificado; en el empírico busca también aprender para decidir qué hacer a continuación.

3. Recursivo: Repite un conjunto de actividades completas buscando principalmente reducir los riesgos del proyecto. El ejemplo principal es el Modelo en Espiral, que en cada ciclo:

1. Determina objetivos.
2. Identifica y analiza riesgos.
3. Desarrolla y prueba.
4. Planifica el siguiente ciclo.

Es especialmente útil en sistemas grandes, complejos o de alto riesgo.

Ventaja:

- ✓ permite atacar los riesgos importantes antes de comprometer grandes cantidades de recursos.

Desventajas:

- ✗ puede resultar costoso y ser difícil establecer hitos claros
- ✗ requiere capacidad para identificar y analizar correctamente los riesgos

[Criterios para elegir un ciclo de vida](#)

La elección debe realizarse según las necesidades del proyecto y características del producto, considerando:

- nivel de riesgo
- claridad y estabilidad de los requerimientos
- incertidumbre
- conocimientos técnicos del equipo
- plazos y necesidad de entregas tempranas
- posibilidad de validar frecuentemente con el cliente

En términos generales:

1. Secuencial:
 - alta certeza, requerimientos estables, pocas posibilidades de cambio y sin necesidad de entregas parciales.
 - Procesos definidos
2. Iterativo/Incremental
 - incertidumbre, requerimientos volátiles, necesidad de feedback y posibilidad de entregar funcionalidad progresivamente.
 - Procesos definidos o empíricos
3. Recursivo/Espiral
 - sistemas grandes o complejos donde la prioridad principal es reducir y administrar riesgos
 - procesos definidos

[Influencia del ciclo de vida en la administración del proyecto](#)

La elección del ciclo de vida condiciona la forma de planificar, monitorear y controlar el proyecto. No se gestiona igual un proyecto:

- secuencial, donde gran parte del trabajo puede planificarse anticipadamente;
- iterativo, donde el plan se revisa y adapta frecuentemente;
- recursivo, donde la gestión gira fuertemente alrededor de los riesgos.

Por eso, la selección del ciclo de vida debe formar parte de la planificación del proyecto, teniendo en cuenta el contexto y las características del producto.

COMPONENTES DE PROYECTO DE DESARROLLO DE SW

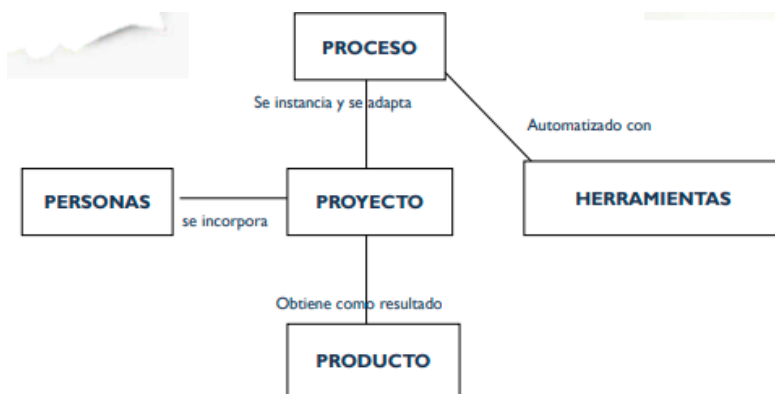
PROYECTO

Esfuerzo temporal que reúne personas y recursos para obtener un producto, servicio o resultado único. Permite adaptar el proceso teórico a las condiciones concretas en las que se realizará el trabajo.

- Único:
 - Nunca hay uno igual a otro. Aunque sean similares tienen características que los diferencian
- Duración limitada:
 - Son temporarios, terminan cuando se alcanza el/los objetivo/s
- Orientado a objetivos:
 - Están dirigidos a obtener resultados y eso se refleja con objetivos, los cuales guían al proyecto. Estos deben ser claros y alcanzables
- Tareas interrelacionadas
 - Basadas en esfuerzos y recursos
 - Permiten manejar la complejidad sistémica de los problemas

Las 4P del desarrollo de SW

1. Personas: quienes participan en el desarrollo. Fundamentales porque el sw es una actividad humano-intensiva.
2. Producto: aquello que se desea construir; sus objetivos, características, alcance y requerimientos.
3. Proceso: marco de actividades, métodos y prácticas que se adapta al proyecto para construir el producto.
4. Proyecto: organiza y coordina personas, proceso y recursos para obtener el producto dentro de determinados objetivos y restricciones.



El **proceso** constituye el marco de trabajo y se instancia y adapta a las características de cada **proyecto**. Al proyecto se incorporan **personas**, el proceso puede ser automatizado o facilitado mediante **herramientas**, y como resultado se obtiene un **producto**.

No existe un único proceso ideal para todos los proyectos: debe adaptarse según el producto, su complejidad, las personas, los recursos y el contexto

¿Por qué es necesario gestionar el software?

Porque presenta una complejidad inherente y es intangible, por lo que su progreso resulta más difícil de visualizar y controlar que el de otros productos físicos. Además, cada proyecto presenta características particulares y los procesos deben adaptarse al contexto.

Más ideas de Brooks sobre el desarrollo de SW

- Más personas no significan necesariamente menos tiempo: existen tareas que no pueden dividirse indefinidamente
- Ley de Brooks: agregar personas a un proyecto de software retrasado puede retrasarlo aún más, porque aparecen costos de capacitación, coordinación y comunicación.
- Al aumentar el equipo, aumentan rápidamente los canales de comunicación:
El esfuerzo de comunicación crece muy rápidamente, reduciendo o incluso anulando las ganancias teóricas de agregar más personas al proyecto.
- Es importante mantener integridad conceptual, es decir, una visión coherente del diseño en lugar de sumar decisiones contradictorias.
- Debe evitarse el efecto del segundo sistema: sobrecargar una nueva versión con funciones y complejidad innecesarias simplemente porque ahora se dispone de más experiencia o recursos.

$$\frac{n(n-1)}{2}$$

Ecosistema de desarrollo exitoso: El éxito no depende solo de la tecnología, sino de combinar:

1. Integridad conceptual: Una arquitectura dictada por una mente maestra, respaldada por estructuras de datos limpias y ocultación de información.
2. Personas empoderadas: contar con equipos pequeños y capacitados, con autonomía y buenas condiciones para trabajar.
3. Proceso orgánico: desarrollar de manera incremental, haciendo crecer el sistema paso a paso y adaptándose continuamente al cambio.

ADMINISTRACIÓN DE PROYECTOS

Consiste en aplicar conocimientos, habilidades, herramientas y técnicas para lograr el trabajo requerido en el tiempo y presupuesto acordados y satisfaciendo los requerimientos (Tener el trabajo hecho, en tiempo y forma, con el presupuesto acordado y habiendo satisfecho los requerimientos)

Incluye:

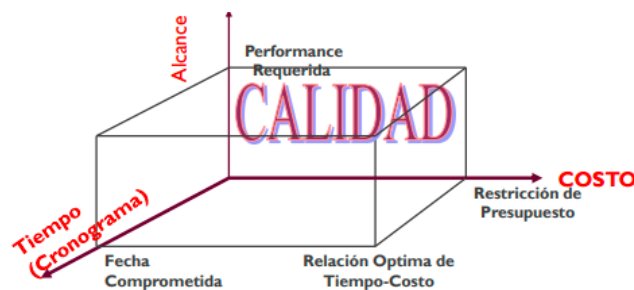
- Identificar los requerimientos
- Establecer objetivos claros y alcanzables
- Adaptar las especificaciones, planes y el enfoque a los diferentes intereses de los involucrados (stakeholders)

TRIPLE RESTRICCIÓN

Todo proyecto debe equilibrar tres variables:

1. Alcance (objetivo): qué debe obtenerse o qué funcionalidades se incluyen.
2. Tiempo: cuánto tiempo se dispone para completar el proyecto.
3. Costo: presupuesto y recursos económicos disponibles.

Estas variables están relacionadas: modificar una puede obligar a ajustar alguna de las otras. El equilibrio entre alcance, tiempo y costo afecta directamente la calidad del resultado. La calidad no debería utilizarse como variable de ajuste; ante una restricción deben negociarse alcance, tiempo o costo.



Líder de proyecto

En un enfoque tradicional, el líder coordina el proyecto, toma decisiones, asigna tareas y mantiene relaciones con los distintos stakeholders. Crea y mantiene el **plan de proyecto** (cambiante)

Equipo de proyecto

Un grupo de personas comprometidas en alcanzar un conjunto de objetivos de los cuales se sienten mutuamente responsables. Un buen equipo:

- reúne conocimientos y habilidades diversos;
- suele ser relativamente pequeño;
- trabaja de forma cohesiva y genera sinergia;
- comparte conocimiento;
- aprende de sus integrantes;
- asume responsabilidad como unidad.

Plan de proyecto

Funciona como la hoja de ruta del proyecto y documenta las principales decisiones de gestión. Debe responder:

- ¿Qué se va a hacer?
- ¿Cuándo se va a hacer?
- ¿Cómo se va a hacer?
- ¿Quién lo va a hacer?

La planificación comprende:

1. Definición del alcance:

- **De producto:** características y funcionalidades del producto o servicio
 - ¿Cómo se mide? Contra especificación de requerimientos
- **De proyecto:** todo el trabajo y solo el trabajo que debe hacerse para entregar el producto o servicio con todas las características y funciones especificadas.
 - ¿Cómo se mide? Contra el plan de proyecto

2. Definición del proceso y ciclo de vida: Se selecciona y adapta el proceso según objetivos, alcance, personas, recursos, cliente y complejidad del producto.

3. Estimación: Forma parte de la planificación y permite obtener aproximaciones sobre tamaño, esfuerzo, calendario, costos y recursos.

4. Gestión de riesgos

Un **riesgo** es un evento incierto que, si ocurre, puede producir una pérdida o afectar el cumplimiento de los objetivos del proyecto. Se identifican y gestionan durante todas las fases del proyecto

La exposición al riesgo Permite evaluar y priorizar riesgos: $\text{Exposición} = \text{Probabilidad} \times \text{Impacto}$

Cuanto mayor sea la exposición, mayor atención requiere el riesgo

Proceso de gestión de riesgos

1. Identificar: reconocer posibles riesgos.
2. Analizar: estimar probabilidad e impacto.
3. Planificar: definir cómo actuar frente a ellos.
4. Monitorear y controlar: revisar continuamente si cambió su probabilidad o impacto

Riesgos frecuentes en software: volatilidad de requerimientos, abandono de integrantes, falta de capacidad del equipo y cambios tecnológicos.

- Plan de mitigación: acciones realizadas antes de que ocurra el riesgo para reducir su exposición.
- Plan de contingencia: acciones previstas para aplicar si el riesgo finalmente ocurre

5. Asignación de responsabilidades: Se definen los roles y responsabilidades de cada integrante. Una misma persona puede cumplir más de un rol dependiendo del tamaño y presupuesto del proyecto.

6. Programación del proyecto: La calendarización lleva la planificación a un nivel más detallado, puede representarse con un Diagrama de Gantt

7. Definición de métricas

Una **métrica** es una medida numérica que brinda visibilidad sobre el estado o avance del proceso, proyecto o producto. Se definen en la planificación y se usan durante la ejecución para monitorear

Las métricas del proyecto se consolidan para crear métricas de proceso que sean públicas para toda la organización del software.

Métricas básicas de un proyecto:

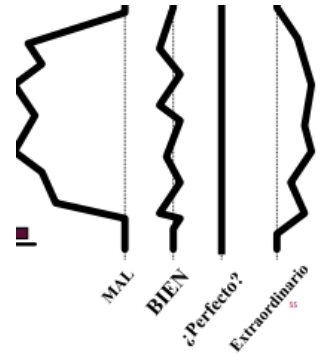
- tamaño del producto y defectos (Producto)
- esfuerzo y tiempo/calendario (Proyecto y proceso, se enuncian de distinta forma)

Las métricas deben mantenerse simples y utilizarse sólo si:

- aportan información nueva
- tienen utilidad práctica
- responden lo que necesitamos conocer

8. Definición de controles: Los controles utilizan las métricas para comparar lo planificado con lo real, detectar desviaciones y decidir acciones correctivas.

Suele no ser perfecto, en el mejor de los casos está bien



Factores principales para el éxito:

1. Monitoreo y feedback.
2. Misión u objetivo claro.
3. Comunicación.

Causas frecuentes de fracaso

- problema mal definido
- planificación basada en información insuficiente
- falta de participación en planificación de quienes harán el trabajo
- ausencia de seguimiento
- plan de proyecto poco detallado
- mala planificación de personas/recursos
- estimaciones basadas sólo en supuestos y sin datos históricos
- ausencia de un responsable claro.

El **proceso** constituye un marco de trabajo que se instancia y adapta al proyecto.

Al **proyecto** se incorporan las **personas**, las actividades pueden facilitarse o automatizarse mediante **herramientas**, y de su ejecución se obtiene el producto de software.

REQUERIMIENTOS ÁGILES – USER STORIES

FILOSOFÍA ÁGIL

El agilísimo no es una metodología o proceso. Es un Pensamiento, una Ideología con un conjunto de principios que guían al desarrollo del producto

El desarrollo ágil busca un equilibrio entre no tener ningún proceso y tener demasiado proceso. Una menor cantidad de documentación puede ser una consecuencia, pero no constituye la esencia de la agilidad. Lo fundamental es que los métodos ágiles son adaptables en lugar de predictivos y están orientados a las personas en lugar de estar orientados al proceso.

Esto tampoco significa trabajar de manera improvisada: ser ágil no es ser indisciplinado. Existe planificación, arquitectura, documentación y procesos, pero se busca que sean suficientes y adaptables, evitando realizar anticipadamente trabajo que todavía no aporta valor.

Algunos frameworks ágiles:

- Scrum
- ShapeUp
- XP
- Crystal
- ATDD

PROCESOS EMPÍRICOS EN AGILE

La agilidad se apoya en gran medida en procesos empíricos. En lugar de intentar predecir todo desde el comienzo, se trabaja en ciclos cortos para obtener resultados, aprender de ellos y adaptar el trabajo. (no deberían superar 1 mes)

Se sostiene en 3 pilares

1. **Transparencia:** la información y el estado real del trabajo deben ser visibles; no ocultar problemas.
2. **Inspección:** observar periódicamente el producto y el proceso para detectar problemas o desviaciones.
3. **Adaptación:** cuando la inspección muestra que algo no funciona o puede mejorarse, actuar y realizar los cambios necesarios.

Funcionan juntos: hacemos visible lo que ocurre → lo inspeccionamos → adaptamos en función de lo aprendido. Esto permite que el equipo vaya adquiriendo experiencia.

En un proyecto ágil el equipo nota que al repetir una iteración con las mismas prácticas obtienen resultados distintos en calidad y velocidad, interpretaciones:

- Es esperable que los resultados varíen aun repitiendo el mismo proceso
- La gestión debe basarse en inspección y adaptación (pilares)
- La predictibilidad absoluta no es un supuesto válido

MANIFIESTO ÁGIL

Descripción de la filosofía del pensamiento ágil. Se valora:

1. **Individuos e interacciones** sobre procesos y herramientas.
2. **Software funcionando** sobre documentación extensiva.
3. **Colaboración con el cliente** sobre negociación contractual.
4. **Respuesta ante el cambio** sobre seguir un plan.

El término “sobre” es fundamental: no significa que procesos, documentación, contratos o planes carezcan de valor, sino que, cuando existe tensión entre ambos lados, se priorizan los primeros

12 Principios del manifiesto



1. Satisfacer al cliente mediante entregas tempranas y frecuentes.

No se espera hasta el final para descubrir si lo construido realmente aporta valor.

2. Aceptar cambios en los requerimientos, incluso en etapas avanzadas.

El cambio no se considera necesariamente un problema; poder responder a él puede convertirse en una ventaja.

4. Técnicos (desarrolladores y diseñadores) y responsables del negocio (product owner) trabajan juntos durante todo el proyecto.

Los requerimientos no se definen una única vez y luego se “entregan” al equipo técnico, hay colaboración continua.

6. La comunicación cara a cara es el medio de comunicación por excelencia.

Permite transmitir información y recibir feedback rápidamente

11. Las mejores arquitecturas, diseños y requerimientos emergen de equipos autoorganizados.

No necesariamente todo puede o debe especificarse completamente al comienzo.

Forma de trabajo en los equipos ágiles

Como consecuencia de esta filosofía, los equipos ágiles:

- trabajan con planificación continua
- son autoorganizados
- realizan entregas frecuentes, iterativas y priorizadas
- aplican prácticas disciplinadas de ingeniería, integración continua y testing concurrente.

Pero trabajar iterativamente no significa simplemente dividir un producto tradicional en etapas. Cada incremento debería aportar funcionalidad utilizable y acercarse al producto deseado.

REQUERIMIENTOS ÁGILES

En Agile se reconoce que no es posible ni conveniente definir todos los requerimientos en detalle desde el inicio, ya que estos pueden cambiar y también surgir nuevos a medida que se aprende sobre el producto.

Se basa en 3 ideas principales:

1. Usar el valor para construir el producto correcto.
2. Usar historias y modelos para mostrar qué construir.
3. Determinar qué es “sólo lo suficiente”, evitando especificar más de lo necesario demasiado pronto.

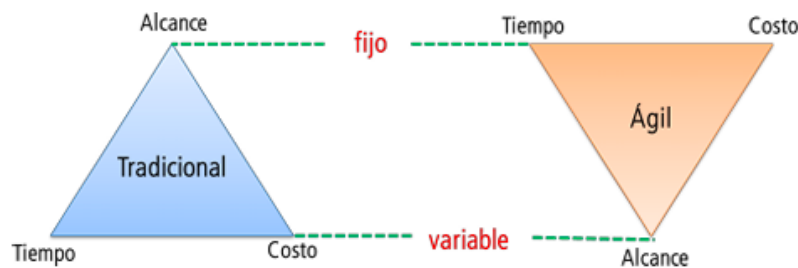
Objetivo: Evitar trabajo innecesario y concentrarse en las funcionalidades que realmente aportan valor al usuario o al negocio

Ingeniería de Requerimientos: comprende dos procesos:

1. Desarrollo de requerimientos: identificar, especificar y validar los requerimientos.
2. Gestión de requerimientos: mantenerlos identificados, íntegros y trazables, gestionando sus cambios durante el desarrollo.

Triple restricción – Enfoque Ágil

A diferencia del enfoque tradicional, donde se fija el alcance y se estiman tiempo y costo, en Agile se acepta que el alcance (requerimientos) puede variar, manteniendo más estables el tiempo y los recursos. Se prioriza qué funcionalidades desarrollar según su valor.

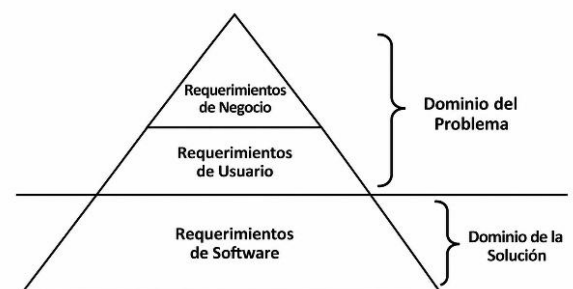


NIVELES:

- **Requerimiento de Negocio:** expresa un objetivo o necesidad del negocio.
Ej.: disminuir el tiempo invertido en procesos manuales de atención al cliente.
- **Requerimiento de Usuario:** expresa una necesidad que el usuario debe poder satisfacer.
Ej.: consultar en línea el estado de cuenta.
- **Requerimiento Funcional:** indica una función o comportamiento que debe proporcionar el sistema.
Ej.: generar un reporte de saldos.
- **Requerimiento No Funcional:** características o restricciones de calidad que debe cumplir el sistema.
Ej.: cumplir determinados niveles de seguridad.
- **Requerimiento de Implementación:** establece condiciones o decisiones sobre cómo implementar la solución. Ej.: utilizar servidores en la nube.

En la gestión ágil el foco está principalmente en los requerimientos de negocio y de usuario.

Las User Stories representan requerimientos de usuario alineados con requerimientos de negocio; no se utilizan para especificar directamente requerimientos de software.



BRUF y requerimientos emergentes

El enfoque tradicional suele asociarse al BRUF (Big Requirements Up Front): intentar relevar y especificar una gran cantidad de requerimientos antes de comenzar el desarrollo. El problema es que se puede invertir mucho esfuerzo en funcionalidades que posteriormente cambien, pierdan prioridad o incluso nunca sean utilizadas.

Solo el 20% de la funcionalidad se usa siempre o frecuentemente

Ágil, en cambio, acepta la existencia de requerimientos emergentes: algunos son conocidos inicialmente, mientras que otros se descubren o modifican a medida que avanza el proyecto y se obtiene feedback.



Análisis JUST IN TIME

Consiste en detallar un requerimiento cuando se aproxima el momento de implementarlo, en lugar de especificar completamente todos los requerimientos al comienzo. Por eso, los elementos próximos a desarrollarse tienen mayor detalle, mientras que los más lejanos pueden permanecer a un nivel más general

PRODUCT BACKLOG

Es el artefacto principal: En él se gestionan los requerimientos y demás necesidades del producto. Es una lista ordenada y dinámica de lo que se necesita realizar para el producto. Sus elementos se refinan y adquieren mayor detalle a medida que se acercan a su implementación.

El Product Owner mantiene la visión del producto y es responsable de orientar y priorizarlo según el valor de negocio

Nunca está 100% completo: durante el proyecto aparecen, cambian o se eliminan requerimientos a medida que se obtiene nueva información y feedback.



Principios Lean relacionados

La gestión ágil de requerimientos se relaciona con tres principios Lean:

1. **Eliminar el desperdicio:** evitar trabajo que no aporta valor, como desarrollar o detallar anticipadamente funcionalidades innecesarias.
2. **Diferir compromisos:** postergar decisiones hasta el momento adecuado, cuando se dispone de mayor información (análisis Just in Time)
3. **Ver el todo:** considerar el producto de manera global y no optimizar requerimientos o partes de forma aislada.

USER STORIES

Técnica para trabajar requerimientos de usuario en ambientes ágiles. Describe brevemente una funcionalidad o necesidad valorada por el usuario y funciona como recordatorio de una conversación futura entre el Product Owner y el equipo. Es un requerimiento a nivel de usuario, no una especificación detallada del software.

Es la descripción corta de una funcionalidad. Qué necesita el usuario de SW

PARTES

1. **Tarjeta (Card):** Contiene una descripción breve de la necesidad (parte visible)
2. **Conversación:** Es la parte fundamental. Permite desarrollar y aclarar los detalles de la historia mediante la comunicación entre el Product Owner, el equipo y los involucrados (parte no visible)
3. **Confirmación:** Acuerdo entre el Product Owner y el equipo que permite demostrar que la funcionalidad fue implementada correctamente. Se materializa mediante las pruebas de aceptación.

Frase verbal (US resumida)

Como [rol], **quiero** [acción/necesidad], **para** [valor de negocio].

- [Rol]: quién necesita la funcionalidad. Debe ser específico, evitar simplemente “como usuario” cuando pueden identificarse roles más concretos.
- [Acción/necesidad]: qué quiere realizar o conseguir ese rol (frase verbal de la US).
- [Valor de negocio]: para qué necesita realizarlo o qué beneficio obtiene. Importante porque permite comprender por qué la funcionalidad es necesaria y ayuda al Product Owner a priorizarla.

Criterios de aceptación (Pruebas de usuario):

- Definen límites para una user story
- Ayudan a que los PO respondan lo que necesitan para que la US provea valor (RF mínimos)
- Ayudan a que el equipo tenga una visión compartida de la US
- Ayudan a desarrolladores y testers a derivar las pruebas.
- Ayudan a los desarrolladores a saber cuándo parar de agregar funcionalidad en una US (Alínean al PO, testers y desarrolladores)
- Buenos criterios:
 - expresan una intención y no una solución
 - son independientes de la implementación
 - se mantienen a un nivel relativamente alto, sin detalles innecesarios de cómo construir la solución.

Pruebas de aceptación

- Expresan detalles resultantes de la conversación
- Complementan la User Story
- Pruebas de (pasa/falla) teniendo en cuenta los criterios de aceptación

Buscar Destino por Dirección	Pruebas de Usuario
Como Conductor quiero buscar un destino a partir de una calle y altura para poder llegar al lugar deseado sin perderme.	▣ Probar buscar un destino en un país y ciudad existentes, de una calle existente y la altura existente (pasa). ▣ Probar buscar un destino en un país y ciudad existentes, de una calle inexistente (falla). ▣ Probar buscar un destino en un país y ciudad existentes, de una calle existente y la altura inexistente (falla). ▣ Probar buscar un destino en un país inexistente (falla). ▣ Probar buscar un destino en País existente, ciudad inexistente (falla). ▣ Probar buscar un destino en un país y ciudad existentes, de una calle existente y demora más de 30 segundos (falla).
Criterios de Aceptación: • La altura de la calle es un número. • La búsqueda no puede demorar más de 30 segundos.	

Una US bien formada:

- Describe una necesidad del usuario
- Tiene valor de negocio explícito
- Reemplaza a la ERS según el estándar de la IEEE

US como elemento multipropósito

No cumple solamente la función de describir una necesidad. Puede funcionar como:

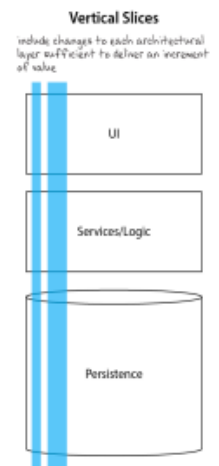
- Una necesidad del usuario
- Una descripción del producto
- Un ítem de planificación
- Token para una conversación
- Mecanismo para diferir una conversación

El Product Owner Prioriza las historias en el Product Backlog

Porciones verticales

Las US deberían representar porciones verticales del sistema, es decir, funcionalidades completas que atraviesen las distintas capas necesarias, como interfaz (GUI), lógica de negocio y base de datos

La idea es NO dividir el trabajo horizontalmente por capas (una historia para hacer la interfaz y otra para la BD) porque c/ historia debería aportar por sí misma una funcionalidad con valor



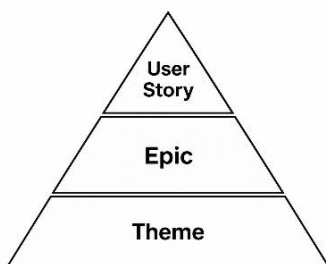
Modelado de usuarios

No conviene escribir todas las historias simplemente desde la perspectiva de “el usuario”, porque un sistema puede tener distintos tipos de usuarios con objetivos y necesidades diferentes.

- **Rol de usuario:** representa un tipo de usuario según sus características, objetivos y relación con el producto. Permite evitar escribir todas las historias desde la perspectiva genérica de “el usuario”.
- **Personas:** Una Persona es una representación ficticia pero concreta de un determinado rol. Se le asignan un nombre y características relevantes para hacer más tangible al usuario y comprender mejor sus necesidades.
- **Usuarios representantes (proxies):** Se usan cuando no es posible acceder directamente a los usuarios reales. Los representan y ayudan a aportar información sobre sus necesidades. Ej: gerentes de usuarios, de desarrollo, persona del grupo de marketing. No es equivalente al usuario real, evitarlos
- **Personajes extremos:** Usuarios ficticios con características/necesidades poco habituales. Se utilizan para analizar el sistema desde perspectivas diferentes y descubrir US que podrían no identificarse pensando en usuarios típicos.

NIVELES DE ABSTRACCIÓN

Las necesidades pueden expresarse con distintos niveles de abstracción:



User Story: unidad más pequeña y concreta, con tamaño adecuado para ser desarrollada.

Epic (Épica): User Story demasiado grande para completarse dentro de una iteración. Debe dividirse en historias más pequeñas antes de implementarse. Puede mantenerse como épica mientras todavía no sea prioritaria.

Theme (Tema): conjunto de User Stories conceptualmente relacionadas, agrupadas para tratarlas como una unidad de planificación.

DEFINITION OF READY (Definición de Listo)

Es una medida de calidad que determina si la User Story está en condiciones de entrar a una iteración de desarrollo. Para que la US pueda ser implementada, mínimamente debe satisfacer el INVEST Model. Entre el equipo de desarrollo y el PO pueden definir características o condiciones extras al INVEST para que una US se considere lista para entrar a la iteración de desarrollo.

MODELO "INVEST"

Una buena US debe cumplirlo

- **I Independent:** Debería poder implementarse y planificarse sin depender fuertemente de otras historias.
- **N Negotiable:** Expresa el qué, no impone el cómo.
- **V Valuable:** Debe aportar valor al cliente/usuario.
- **E Estimable:** debe poder estimarse para considerar su costo/esfuerzo al priorizarla.

Una US puede no ser estimable por tres motivos principales:

1. incertidumbre de negocio
2. incertidumbre técnica
3. es demasiado grande.

En esos casos se debe aclarar la necesidad, investigar mediante un Spike o dividir la historia.

- **S Small:** suficientemente pequeña para poder completarse dentro de una iteración.
- **T Testable:** debe poder comprobarse que fue correctamente implementada.

DEFINITION OF DONE (Definición de Hecho)

Determina si la US está terminada. Son los criterios que establecen cuándo una User Story ha sido implementada de forma correcta y completa, de forma que puede ser mostrada al Product Owner, para que éste decida si se pasa a producción o no.

En la práctica suele expresarse mediante un checklist de condiciones compartido por el equipo, que permite tener una comprensión común de qué significa que el trabajo esté realmente terminado. Generalmente tiene:

- Código completo, refactorizado y con formato estándar
- Código en repositorio e inspeccionado
- Pruebas (unidad, integración, sistema) realizadas
- Cero defectos conocidos y despliegue en producción

SPIKE

Tipo especial de trabajo utilizado para reducir una incertidumbre importante que impide estimar adecuadamente una User Story. Su objetivo es investigar y obtener el conocimiento necesario antes de comprometerse con la implementación; como resultado pueden surgir una o más User Stories.

Se usan para quitar el riesgo e incertidumbre de una US y otra faceta del proyecto. Pueden utilizarse para:

1. Inversión básica para familiarizar al equipo con una nueva tecnología o dominio.
 2. Analizar el comportamiento de una historia compleja y poder dividirla en piezas manejables.
 3. Ganar confianza frente a riesgos tecnológicos, investigando o prototipando.
 4. Ganar confianza frente a riesgos funcionales, donde no está claro cómo el sistema debe resolver la interacción con el usuario para alcanzar el beneficio esperado.
- **Spike Técnicas:** Utilizadas para evaluar diferentes enfoques técnicos y tecnológicos en el dominio de la solución.
 - Evaluar performance potencial
 - Decisión de hacer o comprar
 - Evaluar la implementación de cierta tecnología

Cualquier situación en la que el equipo necesite una comprensión más fiable antes de comprometerse a una nueva funcionalidad en un tiempo fijo. Dependen de los técnicos (nosotros).

- **Spike Funcionales:** Utilizadas cuando hay cierta incertidumbre respecto de cómo el usuario interactuará con el sistema. Usualmente son mejor evaluadas con prototipos para obtener realimentación de los usuarios o involucrados. Dependen de definiciones del negocio, del PO

Lineamientos para spikes:

- Estimables, demostrables, y aceptables
- La excepción, no la regla
 - Toda historia tiene incertidumbre y riesgos
 - Objetivo del equipo: aprender a aceptar y resolver cierta incertidumbre en c/iteración
 - Spikes deben dejarse para incógnitas críticas más grandes
 - Utilizarlas como última opción
- Implementar la spike en una iteración separada de las historias simultaneas.
Salvo que el spike sea chico y sencillo y sea probable encontrar una solución rápida. En ese caso spike e historia pueden incluirse en la misma iteración



ESTIMACIONES DE SOFTWARE

Estimar es obtener una aproximación que permita predecir aspectos como el esfuerzo, costo o duración de un proyecto y ayudar a gestionar sus riesgos.

Consideraciones importantes:

- Por definición, una estimación no es precisa.
- Estimar no es planificar y planear no es estimar: Las estimaciones sirven como base para elaborar planes, pero el plan no tiene por qué coincidir exactamente con lo estimado.
- A mayor diferencia entre lo estimado y lo planificado, mayor riesgo.
- Las estimaciones no son compromisos.

¿Para qué estimamos?

Principalmente para:

1. Predecir la completitud del trabajo.
2. Administrar riesgos.
3. Obtener una aproximación del esfuerzo, costo y cronograma.

Al comienzo del proyecto existe mucha incertidumbre y poca información, por lo que las estimaciones son menos confiables. A medida que el proyecto avanza y aumenta el conocimiento, las estimaciones pueden hacerse más precisas. La incertidumbre no disminuye sólo por el paso del tiempo, sino al tomar decisiones y eliminar fuentes de variabilidad del proyecto.

Fuentes de error

Los errores de estimación pueden surgir por no utilizar una técnica de estimación adecuada o aplicarla de forma incorrecta. También pueden deberse a:

- información imprecisa sobre el software o la capacidad del equipo
- proyecto mal definido o caótico;
- imprecisiones del propio método de estimación;
- actividades omitidas, requerimientos faltantes, documentación, revisiones, testing, reuniones, licencias, etc.

Técnica fundamental: CONTAR

Es la técnica más básica. Consiste en cuantificar elementos del trabajo para poder estimar, por ejemplo, las horas necesarias para construir una funcionalidad.

Métodos de estimación

Basados en: EXPERIENCIA

1. **DATOS HISTÓRICOS:** Utilizar información de proyectos anteriores como referencia para realizar nuevas estimaciones. Permite aprovechar la experiencia acumulada de la organización en lugar de depender únicamente del conocimiento de una persona.
El problema es que no existen dos proyectos exactamente iguales, por lo que es difícil determinar hasta qué punto un proyecto anterior puede utilizarse como base para uno nuevo. La experiencia previa sirve como referencia, pero no puede extrapolarse directamente. Pueden utilizarse:
 - Industry Data: datos de otras organizaciones o del mercado.
 - Historical Data: datos de proyectos anteriores de la propia organización.
 - Project Data: datos obtenidos de etapas anteriores del mismo proyecto.

2. **JUICIO EXPERTO:** Es un método de estimación basado principalmente en el conocimiento y experiencia de una o más personas expertas. Es uno de los enfoques más utilizados en la práctica

- **PURO:** Un experto analiza las especificaciones y realiza una estimación según su experiencia y conocimiento. Puede generar problemas porque es necesario:
 - Definir quién es realmente el experto
 - Convalidar sus conocimientos y experiencia.
 - Evitar que toda la capacidad de estimación dependa de una sola persona; si el experto deja la organización, se pierde gran parte de ese conocimiento.

Por eso, aunque el juicio experto es muy utilizado, conviene estructurarlo para disminuir la subjetividad, se recomienda trabajar con tareas de granularidad aceptable, emplear un checklist con criterios definidos para asegurar cobertura y utilizar un **método optimista, habitual y pesimista**: El experto proporciona tres estimaciones:

- Optimista: mejor escenario razonable.
- Habitual: escenario más probable o normal.
- Pesimista: peor escenario razonable.

Luego se calcula una estimación ponderada:
$$Estimación = \frac{O + 4H + P}{6}$$

Más peso al escenario habitual, pero se consideran posibles desviaciones favorables y desfavorables.

- **WIDEBAND DELPHI:** Es una variante grupal del juicio experto que busca llegar a una estimación consensuada y reducir la dependencia de la opinión de una sola persona. Proceso:
 - Se entregan las especificaciones a un grupo de expertos.
 - Los expertos analizan y discuten el producto y el trabajo a realizar.
 - Cada uno realiza su estimación individual y la entrega a un coordinador.
 - El coordinador comparte las estimaciones de forma anónima.
 - El grupo vuelve a discutir las diferencias.
 - Cada experto revisa su propia estimación.
 - El proceso se repite hasta alcanzar una convergencia razonable.

Es una de las bases conceptuales del Planning Poker, que después se utiliza en estimaciones ágiles.

3. **ANALOGÍA:** Consiste en realizar una estimación comparando el proyecto o trabajo actual con otros proyectos similares ya conocidos. Es un método que tiene mucho error dado que el conocimiento en un proyecto no es extrapolable a otro. Consiste en tomar a un proyecto e ir comparando factores, por ejemplo:
 - Tamaño: si el nuevo proyecto es mayor o menor que uno anterior.
 - Complejidad: si presenta mayor o menor dificultad.
 - Cantidad de usuarios: puede influir en aspectos como concurrencia o volumen de datos.
 - Otras características que permitan establecer semejanzas entre ambos proyectos

Basados en: RECURSOS

Consiste en analizar el personal que se tiene y de cuánto tiempo se dispone del mismo, haciendo las estimaciones exclusivamente en función de este factor

Basado en: EL MERCADO

Ponen el foco en el cliente y realizan las estimaciones en función de su capacidad de pago. En este método no se evalúa al producto que se está intentando vender, sino a quien se lo vende

Basados en: COMPONENTES DEL PRODUCTO o en el PROCESO DE DESARROLLO

se pueden aplicar 2 enfoques:

1. **Bottom-up:** se descompone el producto en unidades lo más pequeñas posible. Se estima cada unidad y luego se hace la estimación total.
2. **Top-down:** se descompone el producto en grandes bloques y luego se estima cada uno de los componentes.

MÉTODOS ALGORÍTMICOS

Se ejecutan algoritmos que tienen como parámetros de entrada medidas cuantitativas de tamaño, complejidad y conocimiento de dominio. Cada variable tendrá asociada un factor multiplicativo y en función de esto se obtiene como resultado un valor de esfuerzo (puede no ser certero)

Estimaciones tradicionales

Se busca estimar distintas variables del proyecto mediante medidas absolutas:

1. **Tamaño:** cuánto producto o software debe construirse.
2. **Esfuerzo:** cantidad de trabajo necesario para desarrollarlo.
3. **Calendario:** duración o tiempo necesario para completar el proyecto.
4. **Recursos:** personas y demás recursos necesarios para realizarlo.

Como el alcance se considera fijo, primero se estima el tamaño del producto y, a partir de este, se derivan las demás estimaciones.

Problemas:

- **Estimar demasiado pronto:** se intenta estimar cuando hay poca información y mucha incertidumbre
- **Resistencia a las reestimaciones:** se mantiene la estimación inicial, aunque aparezca información nueva
- **Búsqueda excesiva de precisión:** se pretende obtener valores demasiado exactos, aunque toda estimación tenga incertidumbre.
- **Estimaciones hechas por personas distintas de quienes harán el trabajo:** quien estima puede no conocer las condiciones reales de ejecución.
- **Estimaciones demasiado optimistas:** las personas tienden a subestimar el trabajo necesario y a asumir escenarios favorables, especialmente al comienzo del proyecto, cuando todavía hay mucha incertidumbre.

ESTIMACIONES EN AMBIENTES ÁGILES

En Agile, uno de los principales beneficios de las estimaciones es el proceso de hacerlas: obliga al equipo a analizar y discutir el trabajo, permite detectar tempranamente si lo planificado es factible y puede servir como protección para el equipo.

El tiempo de la iteración y los recursos se mantienen fijos, mientras que el alcance puede ajustarse. Por eso la estimación busca determinar cuánto trabajo puede realizar el equipo dentro de una iteración.

Matriz de desambiguación:

- **Objetivo (Target):** meta de negocio deseada. Naturaleza: Subjetivo
- **Estimación (Estimate):** predicción analítica e imparcial del esfuerzo o costo. Naturaleza: Subjetivo
- **Compromiso (Commitment):** promesa de entrega que asume determinados riesgos. Naturaleza: Acordado

Se negocia el compromiso, no la estimación: si la estimación no coincide con el objetivo de negocio, pueden ajustarse alcance, fecha o riesgos, pero no debería modificarse artificialmente la estimación.

Buena estimación:

- se expresa preferentemente mediante rangos, no como un único valor (12 a 16 semanas)
- explicita un nivel de probabilidad o confianza (con un 75% de confianza)
- se mantiene separada del objetivo de negocio (no fue modificada para encajar en una fecha)
- permite tomar decisiones de control (visibilidad para negociar alcance, recursos o fechas)

Dinámica destructiva de la subestimación (ciclo)

Una estimación artificialmente baja no acelera el desarrollo, sino que termina recortando actividades fundamentales y hace que los errores se descubran en etapas más costosas del proyecto.

1. Cronograma irreal dictado por el negocio.
2. Se recortan actividades tempranas, como análisis y diseño.
3. Se programan soluciones rápidas y “sucias” (Quick & Dirty).
4. Aumenta la cantidad de defectos detectados en pruebas o producción.
5. Se generan reuniones de crisis, refactorización tardía y desvío de personal.

El apuro no acelera realmente el desarrollo; desplaza los problemas hacia etapas posteriores, donde corregirlos resulta más costoso.

ESTIMACIÓN RELATIVA

Las US se estiman con medidas relativas, comparándolas entre sí en lugar de intentar determinar valores absolutos.

Las personas suelen ser mejores comparando que estimando en términos absolutos. Por ejemplo, resulta más sencillo determinar que una historia es aproximadamente el doble de grande que otra que afirmar exactamente cuántas horas llevará.

La estimación relativa, además:

- es más rápida
- favorece la discusión del equipo
- genera pensamiento colectivo
- permite aprovechar mejor el tiempo destinado al análisis.

TAMAÑO DE UNA USER STORY

El tamaño representa cuán grande es el trabajo necesario para completar una User Story. Para estimarlo se consideran principalmente tres aspectos:

- **Complejidad:** dificultad propia de la funcionalidad.
- **Esfuerzo:** cantidad de trabajo necesaria para completarla.
- **Incertidumbre:** falta de información, ya sea técnica o de negocio.

El tamaño y el esfuerzo no son lo mismo

El tamaño es una medida relativa de la User Story, mientras que el esfuerzo depende de quién realiza el trabajo y puede variar según sus habilidades, conocimientos, experiencia, supuestos y familiaridad con el dominio. Por eso, una misma User Story puede tener el mismo tamaño para el equipo, aunque implique distinto esfuerzo para diferentes personas.

Story Point (SP)

Unidad de medida relativa y propia de cada equipo que expresa el “peso” o tamaño relativo de una User Story considerando complejidad, esfuerzo e incertidumbre

No representan horas ni tiempo calendario. Lo importante no es el valor absoluto asignado, sino que las estimaciones sean consistentes entre las historias del mismo equipo.

Para expresar tamaños relativos pueden utilizarse distintas escalas:

- Números del 1 al 10.
- Talles de remera: S, M, L, XL, etc.
- Serie exponencial: 1, 2, 4, 8, 16...
- Serie de Fibonacci: 1, 2, 3, 5, 8, 13, 21...

Una vez elegida una escala, debe mantenerse para conservar un mismo patrón de comparación.

Fibonacci: Se utiliza porque, a medida que aumenta el tamaño también aumenta la incertidumbre. Por eso los valores de la escala se van separando progresivamente. En términos generales:

- 0: no se posee información suficiente para estimar.
- ½, 1: funcionalidad pequeña
- 2, 3: funcionalidad pequeña a mediana.
- 5: funcionalidad mediana.
- 8: funcionalidad grande; conviene evaluar si puede dividirse.
- 13 o +: normalmente indica que la historia es demasiado grande y debería dividirse antes de ingresar a una iteración.

Un PO exige convertir Story Points directamente a horas para comprometer fechas, genera problemas como:

- Confusión entre tamaño y esfuerzo
- Uso indebido de las estimaciones
- Precisión indebida sobre el equipo
- Pérdida del enfoque empírico

VELOCIDAD

Es una métrica del progreso de trabajo del equipo. Se calcula sumando los Story Points de las User Stories completadas durante una iteración (US desarrollada y testeada por completo, aceptada por el PO).

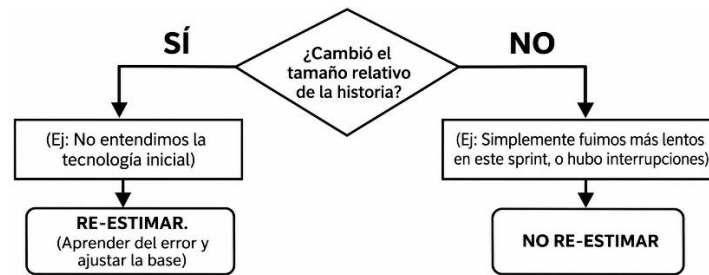
Solo se cuentan historias terminadas; una historia parcialmente completada aporta 0 puntos a la velocidad de esa iteración. La velocidad permite:

- conocer aproximadamente cuánto trabajo puede completar el equipo en futuras iteraciones. Conviene observarla a lo largo de varias iteraciones y no concentrarse en el valor aislado de un único Sprint.
- retroalimentar y corregir las inexactitudes del proceso de estimación a lo largo de las iteraciones.

Reestimación

Una User Story debería reestimarse cuando cambia la percepción de su tamaño relativo, por ejemplo, porque al compararla con nuevas historias se descubre que era mayor o menor de lo pensado.

No debería modificarse su estimación simplemente porque el equipo avanzó más lento o más rápido de lo esperado. Esas diferencias se reflejan y compensan mediante la velocidad del equipo a lo largo de sucesivas iteraciones.



Método de estimación: POCKER PLANNING

técnica grupal de estimación en la que los miembros del equipo estiman las User Stories mediante comparación. Combina:

- juicio experto
- analogía
- desagregación.

Las estimaciones las realizan quienes desarrollarán el producto (las personas más competentes en resolver una tarea). El Product Owner puede participar aclarando dudas sobre las historias, pero no asigna los Story Points.

Prerrequisitos:

- Lista de user stories a ser estimadas
- Un mazo de cartas por estimador

User Story canónica: historia simple y bien comprendida que se utiliza como patrón de referencia para comparar el tamaño de las demás. Se le asigna normalmente 1 o 2 Story Points.

Procedimiento:

1. Se selecciona una historia de referencia o **canónica** y se le asigna un tamaño conocido.
2. Se presenta la historia que se desea estimar y se aclaran las dudas con el Product Owner. Esta discusión debe ser breve y timeboxed.
3. Cada integrante selecciona en privado una carta con su estimación.
4. Todos muestran las cartas simultáneamente, evitando que una opinión influya inicialmente sobre las demás.
5. Si todos los estimadores selecciona el mismo valor, ese es el estimado.
Si existen diferencias, quienes eligieron los valores más alto y más bajo explican sus razones.
6. Se vuelve a estimar hasta alcanzar una estimación razonablemente consensuada.

En Agile no se estima directamente el tiempo. Primero se estima el tamaño mediante Story Points, luego se observa la velocidad del equipo y a partir de ambas se deriva la duración (cronograma)

$$\text{Duración aproximada en iteraciones} = \frac{\text{Story Points totales}}{\text{Velocity}}$$

STORY POINTS vs DÍAS IDEALES

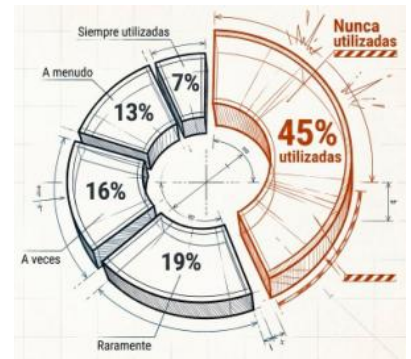
- **Story Points:** representan tamaño y complejidad relativa y no dependen directamente del tiempo.
- **Días ideales:** representan cuánto tiempo llevaría el trabajo sin interrupciones y pueden variar según la experiencia o velocidad de quien lo realiza.

Mejor usar SP porque permiten separar mejor tamaño y duración y favorecen una estimación colectiva del equipo.

GESTIÓN DE PRODUCTO

PRODUCTO DE SOFTWARE

Artefacto construido para satisfacer una necesidad específica a partir de ciertos requerimientos. Sin embargo, crear más funcionalidades o desarrollar más rápido no significa necesariamente generar más valor. El mayor riesgo es construir correctamente un producto que nadie quiera utilizar.

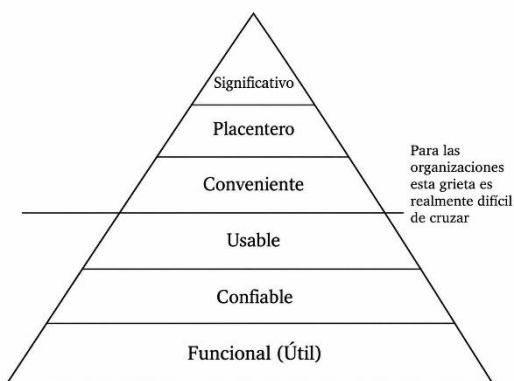


Valor del producto:

El foco debe estar en construir lo correcto, lo que realmente aporta valor al usuario.

No depende solamente de que el producto sea funcional. La experiencia del usuario adquiere cada vez mayor importancia: el producto debe resolver una necesidad, pero también resultar confiable, usable y proporcionar una experiencia que motive a utilizarlo. En mercados competitivos, ser simplemente viable puede no ser suficiente; el objetivo puede evolucionar hacia crear un producto memorable o “adorable”.

Evolución del producto:



Primero el producto debe realizar correctamente su función; luego debe ser confiable y permitir que el usuario alcance sus objetivos eficientemente.

Los niveles superiores buscan que el producto facilite realmente la vida del usuario y genere una experiencia positiva y significativa.

LEAN STARTUP

Busca reducir el riesgo de construir algo que el mercado no necesita mediante un ciclo continuo:

Se construye algo pequeño, se observa cómo reaccionan usuarios reales, se aprende de los resultados y se utiliza ese aprendizaje para modificar o continuar el producto. El objetivo es equivocarse y aprender temprano, cuando hacerlo todavía resulta barato

La productividad no debe medirse por cuánto se construye, sino por cuánto aprendizaje y valor se obtiene. Si algo no genera valor ni aprendizaje, representa desperdicio

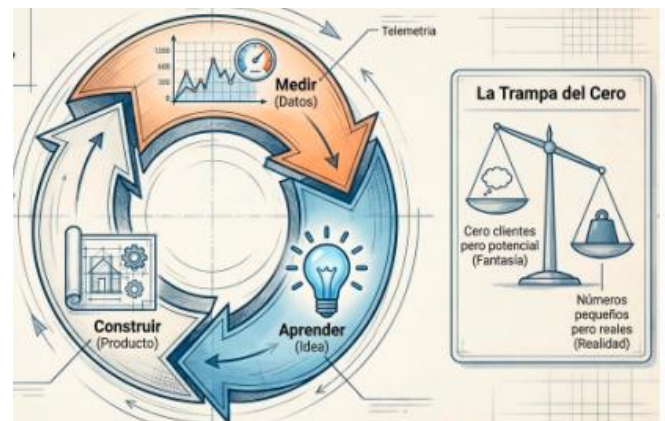
Rompe la parálisis de la audacia del cero

La “Audacia del Cero” / Trampa del Cero

Cuando un producto todavía no fue lanzado, puede tener 0 clientes y 0 ingresos, pero también 0 evidencia real de fracaso. Ese “cero” puede resultar engañosamente cómodo, porque permite seguir suponiendo que el producto tendrá éxito cuando finalmente esté “perfecto”.

El problema es que, por miedo a mostrar resultados pobres o lanzar algo incompleto, se puede postergar demasiado el lanzamiento. Esto genera una falsa sensación de seguridad, pero en realidad aumenta el riesgo: se invierte más esfuerzo sin feedback real y se puede terminar construyendo algo que nadie quiere.

El antídoto es salir temprano con un MVP, observar el comportamiento de usuarios reales, medir y aprender. El objetivo es confrontar la hipótesis con la realidad lo antes posible y evitar la sobreconstrucción.



1. PRODUCTO MÍNIMO VIABLE – MVP “mínimum viable product”

Experimento utilizado para validar hipótesis sobre un nuevo producto con el menor esfuerzo posible, obteniendo aprendizaje de usuarios reales (feedback) lo antes posible.

Su finalidad principal no es ganar dinero ni constituir el producto definitivo, sino aprender si estamos construyendo el producto correcto. Se construye sólo lo necesario hasta comprobar que existe un interés genuino en el producto. Esto evita invertir tiempo y recursos en funcionalidades que podrían no utilizarse

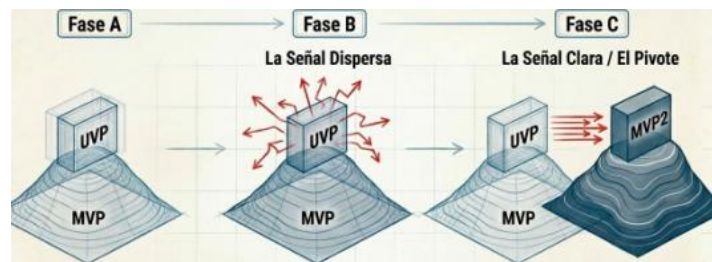
- Un MVP puede ir desde una prueba de humo (todavía no existe un producto construido, por ejemplo, un video demostrando la tecnología) hasta un prototipo temprano o una versión limitada del producto
- Sirve para proveer una dosis necesaria de realidad
- Maximiza el aprendizaje validado y minimiza el esfuerzo inicial

PROPUESTA DE VALOR ÚNICA “UVP”

Representa la hipótesis principal acerca del valor diferencial que ofrecerá el producto. El MVP se construye para ponerla a prueba

Si distintos usuarios piden funcionalidades totalmente diferentes para poder utilizar el producto, probablemente todavía no se encontró un mercado claro. Si muchos usuarios comienzan a señalar la misma necesidad, aparece una señal que permite reformular la hipótesis y realizar un pivot, construyendo un nuevo MVP(2) basado en lo aprendido

Pivotar: modificar la hipótesis de producto, cliente o solución utilizando el aprendizaje obtenido previamente.



¿Qué debe tener un MVP?

Debe enfocarse en los elementos críticos de la propuesta de valor, agregando únicamente aquello indispensable para que pueda utilizarse y evaluarse. Centrarse principalmente en los componentes urgentes e importantes



MVP como “Feta vertical”:

Un MVP no debería construirse terminando completamente una capa del producto y dejando las demás sin desarrollar. Debe basarse en la propuesta de valor única “UVP” y ofrecer lo mínimo e indispensable para ser viable

Debe ser una feta vertical y cohesiva que contenga, aunque sea en una cantidad mínima:

- Funcionalidad + Fiabilidad + Usabilidad + Diseño

Es preferible tener un producto limitado pero usable de extremo a extremo antes que tener una única dimensión perfectamente desarrollada.

Hipótesis que permite validar:

1. **Hipótesis del valor:** prueba si el producto realmente está entregando valor a los clientes después de que comienzan a usarlo (ver si el producto es realmente el producto correcto).
Si se lanza una feature sin validarla se aumenta el riesgo de: desperdicio, construir algo que nadie quiere, sobre-ingeniería
→ Métrica: tasa de retención, más de la mitad de los usuarios volvieron a utilizar el sistema todos los días, solo un 25% dejó de usarlo
2. **Hipótesis de crecimiento:** prueba cómo nuevos clientes descubrirán el Producto (ver si más personas desean usar el producto).
→ Métrica: tasa de referencia (Se corrió la voz acerca de lo que era, no fue necesario el uso de políticas de marketing o publicidad).

2. PRODUCTO MÍNIMO COMERCIALIZABLE – MMF “minimum marketable feature”

Es la pieza más pequeña de funcionalidad que puede liberarse aportando valor comercial concreto.

Se utiliza cuando existe una alta certeza de que esa área genera valor. Su objetivo es aportar valor rápidamente y mejorar el Time to Market.

- Si al liberarlo te sentís cómodo trabajando con el próximo MMF del área, estas en el camino correcto

3. CARACTERÍSTICA MÍNIMA VIABLE – MVF “minimum viable feature”

Característica pequeña utilizada para probar una hipótesis dentro de un producto que ya existe.

Es similar a aplicar el concepto de MVP sobre una única funcionalidad: MVF = versión mini del MVP

Se utiliza principalmente cuando existe alta incertidumbre respecto de una nueva característica y queremos aprender cómo reaccionan los usuarios antes de desarrollarla completamente.

Debe proporcionar valor suficiente para ser utilizada por usuarios reales y permitir obtener feedback que guíe decisiones futuras.

Reduciendo el riesgo de ejecución

Aunque MVP, MMF y MVF sean unidades mínimas desde el punto de vista de negocio, pueden seguir siendo grandes desde el punto de vista de implementación. Por eso se dividen en User Stories pequeñas, que permiten reducir el riesgo técnico y de ejecución y obtener retroalimentación frecuente.

Ecosistema funcional para escalar: Una vez logrado el encaje producto-mercado, el crecimiento se diseña combinando MMFS (para explotar certezas) y MVFs (para explorar nuevas áreas de valor).

4. MMP “minimum marketable product”

Es el primer producto comercializable dirigido generalmente a los early adopters. Contiene las características esenciales necesarias para comenzar a competir en el mercado. Es el 1° reléase comercial (MMR1)

Relación MVP–MMP

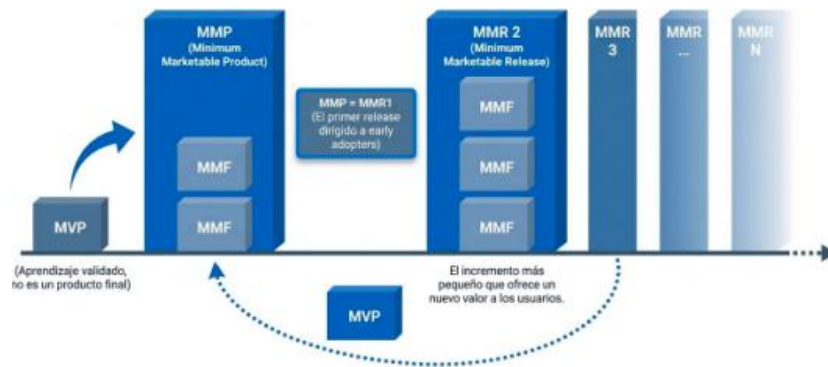
MVP y MMP no son sinónimos porque persiguen objetivos diferentes.

- El MVP busca principalmente validar hipótesis y aprender
- El MMP debe estar en condiciones de comercializarse y competir en el mercado.

Sin embargo, pueden coincidir: un MVP podría ser también MMP si, además de permitir validar las hipótesis, posee el nivel mínimo de funcionalidad, fiabilidad, usabilidad y robustez necesario para comercializarlo

5. MMR “mínimum marketable release”

Es el incremento más pequeño del producto que puede liberarse aportando nuevo valor al usuario. Un MMR puede contener distintas MMF.



1. **Aprendizaje (MVP):** iteraciones rápidas para validar hipótesis.
2. **Lanzamiento (MMP/MMR1):** El primer release orientado al mercado (producto robusto, utilizable)
3. **Evolución (MMR2, MMR3 ...):** releases sucesivos que agregan nuevas características de valor

6. MLP “minimum lovable product” (memorable, adorable)

Es un producto que, además de ser útil y viable, busca generar una conexión positiva con el usuario desde las primeras versiones. No alcanza con que el producto funcione: el objetivo es que el usuario quiera usarlo, lo valore y prefiera seguir utilizándolo frente a otras alternativas. Busca:

- **Cuidar la reputación de marca:** producto debe transmitir atención real por el usuario y reforzar una imagen positiva de la marca.
- **Destacar en el mercado:** en contextos competitivos, no alcanza con ser simplemente funcional o viable. El producto debe diferenciarse, captar la atención del usuario y ofrecer algo que resulte atractivo frente a otras opciones.
- **Generar un bucle de retroalimentación leal:** al ofrecer una experiencia positiva desde el inicio, aumenta la probabilidad de recibir feedback valioso de usuarios comprometidos, que ayude a mejorar y evolucionar el producto.

10 bloques de construcción de la adorabilidad:

- Base — Utilidad:

Esperanza, Satisfacción y Cuidado.

Son los elementos más básicos: el producto debe resolver problemas reales, generar una experiencia satisfactoria y demostrar atención por el usuario.

- Nivel intermedio — Crecimiento:

Confianza en uno mismo, Confianza en el sistema, Escala y Sostenibilidad.

Permiten que el usuario siga utilizando el producto, confiando en él y creciendo junto con su uso.

- Nivel superior — Inspiración:

Motivación y Diversión.

Hacen que el producto no sólo sea útil, sino también atractivo y agradable de usar.

- Cima:

Halo.

Efecto global positivo que hace que el producto resulte especialmente valioso, memorable o deseable.

	MVP (Viable)	MMP (Comercializable)	MLP (Adorable)
Objetivo Principal	Validar concepto de forma económica	Atender a base amplia con funcionalidad clave	Crear conexión emocional e interacción profunda
Enfoque de Diseño	Investigar y empatizar	Construir y refinar / Código y testing	Diseñar para el deleite / Impacto emocional
Beneficio Principal	Informar el alcance del producto	Asegurar la calidad y salida al mercado	Aumentar la lealtad y el amor del usuario
Ejemplo Típico	Entrevistas y videos	Lanzamiento Beta	App con animaciones UI divertidas y personalizadas

Mapeando la estrategia sobre el ciclo de vida del producto

1. Dominio del MVP: maximizar aprendizaje y reducir riesgo del cero
2. Dominio del MMP y MMF: incrementar valor comercial concreto y lograr salida al mercado.
3. Dominio del MLP: crear deleite emocional continuo para evitar la deserción de usuarios.

Cuando aparecen nuevas incertidumbres o nuevas necesidades, el ciclo Lean se reinicia, pudiendo:

- Pivotar
- Crear nuevos MVP / MVF para seguir aprendiendo

FRAMEWORK SCRUM

Scrum es un framework liviano que ayuda a personas, equipos y organizaciones a generar valor mediante soluciones adaptativas para problemas complejos. Es deliberadamente incompleto: establece una estructura mínima de roles, eventos, artefactos y compromisos, dentro de la cual pueden aplicarse distintas técnicas, procesos y prácticas según el contexto.

se apoya en:

- Empirismo: las decisiones se basan en experiencia, observación y evidencia.
- Pensamiento Lean: reducir desperdicio y enfocarse en lo esencial.
- Enfoque iterativo e incremental: trabajar mediante ciclos cortos que permitan aprender, entregar valor y adaptarse.

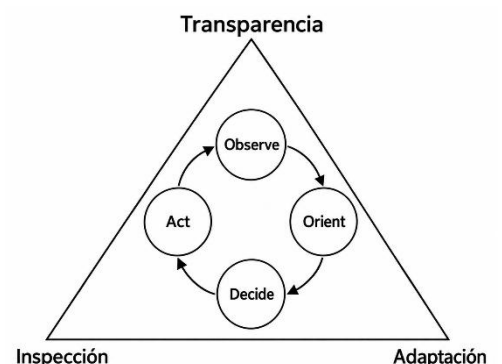
Principios

1. Control empírico del proceso.
2. Autogestión.
3. Colaboración.
4. Priorización basada en valor.
5. Timeboxing.
6. Desarrollo iterativo

El motor: Emiprismo y ciclo OODA

En la expansión se agrega el nuevo concepto de OODA, los pilares se mantienen.

OODA: Forma de representar la toma de decisiones y adaptación continua frente a la incertidumbre:



- **Observar:** reunir evidencia sobre la realidad del producto, el trabajo y el entorno.
- **Orientar:** interpretar esa evidencia, cuestionar supuestos y comprender la situación.
- **Decidir:** seleccionar qué acción o experimento realizar.
- **Actuar:** ejecutar la decisión y obtener nuevos resultados que vuelven a ser observados.

Relación con los pilares del empirismo

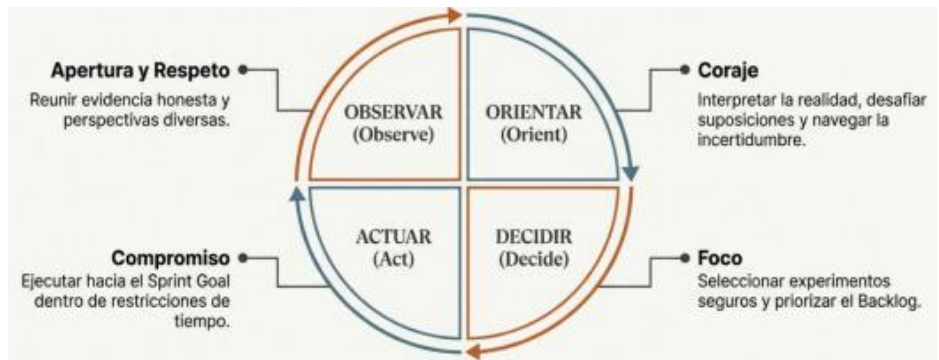
- Transparencia → permite observar la realidad.
- Inspección → permite analizar y detectar variaciones.
- Adaptación → permite modificar la acción a partir de lo aprendido.

Valores

Scrum se apoya en cinco valores:

Compromiso, Foco, Apertura, Respeto, Coraje.

Estos valores favorecen la confianza, la colaboración y el funcionamiento del empirismo.



SCRUM TEAM como unidad Autogestionada

Son quienes ejecutan el ciclo de aprendizaje. Se apoya en 3 características

1. **Autogestión**: el equipo decide quién hace qué, cómo, cuándo y dónde, sin microgestión externa.
2. **Profesionalismo**: responsabilidad sobre el producto y excelencia técnica y ética.
3. **Diversidad cognitiva**: combinación de habilidades y perspectivas para resolver problemas complejos.

La idea es que Scrum no funciona sólo por tener eventos y artefactos: necesita un equipo empoderado capaz de observar, decidir y adaptarse por sí mismo.

ROLES

1. Product Owner: Es responsable de maximizar el valor de largo plazo del producto y de gestionar eficazmente el Product Backlog. Entre sus responsabilidades:

- definir y comunicar el Product Goal;
- crear y ordenar los ítems del Product Backlog;
- asegurar que el Product Backlog sea transparente y comprendido;
- interactuar con stakeholders;
- evaluar trade-offs y orientar el producto hacia la generación de valor.

Es una persona, no un comité. Puede colaborar o delegar trabajo, pero mantiene la responsabilidad final.

2. Product Developers: Son quienes crean los Incrementos utilizables del producto. Son responsables de:

- crear y actualizar el Sprint Backlog;
- mantener la calidad;
- producir al menos un Increment utilizable por Sprint;
- adaptar diariamente el plan para alcanzar el Sprint Goal;
- aprender a partir de la evidencia obtenida;
- responsabilizarse colectivamente por el trabajo.

En la expansión 2026, puede ser humano o automatizado, aunque debe existir al menos un Product Developer humano. La IA puede ampliar las capacidades del equipo, pero debe mantenerse la responsabilidad humana sobre los resultados.

3. Scrum Master: Es responsable de la efectividad de Scrum y actúa como líder servicial y agente de cambio. Ayuda:

- comprender y aplicar Scrum;
- fomentar autogestión y multifuncionalidad;
- remover impedimentos;
- mejorar continuamente;
- asegurar que los eventos sean productivos y respeten sus timeboxes;
- acompañar al Product Owner, Product Developers, stakeholders y organización.

En Scrum siempre existieron actores externos vinculados al producto, pero la Expansión de 2026 los explicita y les da un lugar más formal dentro del ecosistema de roles.



4. Stakeholder: persona, grupo o entidad interesada, afectada o capaz de influir sobre el producto. Provee necesidades, información, feedback o restricciones que ayudan a determinar el valor que debe generar el producto. Puede incluir clientes, usuarios, decisores, sponsors, reguladores, expertos, etc. Rol externo al SCRUM TEAM

5. Supporter: es un Stakeholder particular que actúa como agente de cambio e intermediario entre el Scrum Team y la organización. Ayuda a remover barreras organizativas, facilita recursos y condiciones de trabajo e interactúa con otros Stakeholders para empoderar al equipo y favorecer su autogestión. Su foco no está en decirle al equipo cómo desarrollar el producto, sino en trabajar sobre el entorno y el proceso para que Scrum pueda funcionar correctamente.

Autogestión e inteligencia aumentada

El Scrum Team funciona como una unidad profesional autogestionada. El equipo decide quién hace qué, cómo y cuándo. No hay jerarquías, solo responsabilidades compartidas.

Al como 'Compañero de Equipo': La Inteligencia Artificial se integra en el descubrimiento y entrega.

Human in the loop: La IA aumenta la capacidad, pero la responsabilidad final permanece 100% en los humanos. Su uso debe ser supervisado y ético, ayudando a validar hipótesis, detectar errores o sesgos y mejorar decisiones, no reemplazando el juicio profesional.

Liderazgo y cambio

En Scrum, el liderazgo es una conducta, no necesariamente un cargo. Su objetivo es crear claridad, confianza y un entorno que permita al equipo autogestionarse y responder ante situaciones emergentes.

Idea de Genchi Genbutsu:

→ “ir, ver, escuchar y entender”. Las decisiones deberían basarse en evidencia y observación directa de la realidad, no solamente en reportes alejados del lugar donde ocurre el trabajo.

Cambio emergente: En problemas complejos no puede definirse de antemano un destino completamente fijo. Se establece una dirección, se observa lo que sucede y se adapta el camino, los líderes actúan como catalizadores

SPRINT

Evento contenedor de Scrum donde las ideas se convierten en valor, es un contenedor “seguro para fallar”, por lo que brinda estabilidad. Tiene una duración determinada de hasta 4 semanas y, cuando termina uno, comienza inmediatamente el siguiente. Durante el Sprint:

- Existe un Sprint Goal (Objetivo del Sprint) que funciona como compromiso y debe mantenerse estable
- El alcance es negociable: a medida que se aprende, pueden aclararse, reemplazarse o renegociarse los trabajos necesarios para alcanzar el objetivo, siempre que no se ponga en riesgo el objetivo
- la calidad se construye en cada Sprint, cada Incremento debe cumplir con la Definition of Output Done (estándar de calidad). no existe un “Sprint de hardening” posterior para agregar calidad.
- el Product Backlog se refina cuando es necesario;

Sprints + cortos permiten obtener feedback + frecuente, generar + ciclos de aprendizaje y limitar el riesgo.

EVENTOS DE SCRUM

Scrum combina 4 eventos timeboxed de inspección y adaptación dentro del Sprint.

Timebox se definió considerando un sprint de 1 mes = 4 semanas

1. Sprint Planning (diseño táctico): Es el primer evento del Sprint y define el trabajo inicial. Responde tres preguntas:

- ¿Por qué? definir el Sprint Goal (Objetivo).
- ¿Qué? seleccionar Product Backlog Items.
- ¿Cómo? los Product Developers determinan cómo realizar el trabajo, plan técnico de ejecución

Los Product Developers deciden cuánto trabajo pueden asumir considerando desempeño anterior, capacidad disponible y Definition of Output Done. No conviene sobrecargar al equipo; puede dejarse holgura para manejar incertidumbre.

→ Timebox: máximo 8 horas

2. Daily Scrum (meeting) (navegación táctica): Reunión diaria de los Product Developers cuyo propósito es inspeccionar el progreso hacia el objetivo y adaptar el Sprint Backlog.

- favorece sincronización, autogestión y decisiones rápidas;
- permite detectar riesgos e impedimentos;
- no es un reporte de estado para el Scrum Master.

Los Product Developers pueden utilizar cualquier estructura que les resulte útil.

→ Timebox: 15 minutos

3. Sprint Review (validación de valor): Es una sesión de trabajo colaborativa con los stakeholders para inspeccionar el resultado del Sprint y decidir posibles adaptaciones.

- Se inspecciona el incremento real
- Se valida contra la Definition of Outcome Done (¿resolvimos el problema del usuario?)

No debe ser una simple presentación o demostración

→ Timebox: máximo 4 horas

En uno el PO rechaza varias US porque “no era lo que esperaban” aunque el equipo afirma haberlas completado. ¿Qué problemas estructurales pueden haber ocurrido?

- Definition o Done DoD poco clara o inexistente
- Definition of Ready DoR con problemas
- Falta de transparencia en el sprint
- Criterios de aceptación insuficientes

4. Sprint Retrospective: Busca mejorar la calidad y efectividad de la forma de trabajar.

Las mejoras importantes deben implementarse lo antes posible, incluso dentro del próximo sprint

→ Timebox: máximo 3 horas

Refinamiento del product backlog: Actividad continua que busca reducir incertidumbre y aumentar la claridad de los Items futuros del Product Backlog. Puede realizarse informalmente o estructurarse como una reunión, pero no forma parte de los cuatro eventos principales contenidos en el Sprint.

→ Timebox: aproximadamente 10 % del tiempo de Sprint

Timebox

Establece la cantidad máxima de tiempo disponible para un evento. No significa que deba utilizarse necesariamente todo ese tiempo. El timeboxing ayuda a mantener foco y obliga a adaptar principalmente el alcance, en lugar de extender indefinidamente tiempo o costos.

Discovery y Delivery

En Scrum, Discovery y Delivery no son fases separadas: pueden ocurrir dentro del mismo Sprint.

- Discovery: descubrir y validar necesidades, problemas o posibles soluciones.
- Delivery: construir y entregar Incrementos utilizables.

El objetivo es aprender y entregar valor de forma continua.

ARTEFACTOS Y COMPROMISOS

Artefacto	Compromiso
Product Backlog Lista emergente y ordenada de ítems para avanzar hacia el Product Goal. Única fuente de trabajo del Scrum Team	Product Goal (Objetivo Producto) Es el objetivo estratégico de mediano plazo que da dirección al producto y al Product Backlog.
Sprint Backlog Es un plan creado por y para los Developers y se actualiza durante el Sprint a medida que aparece nueva información.	Sprint Goal Es el objetivo unificador del Sprint, creado durante el Sprint Planning. Da foco al equipo y debe ser coherente con el Product Goal
Increment (del producto) Ladrillo concreto de valor agregado	Definition of Output Done (DoD) Estandar de calidad requerido para liberar

Dimensiones de “Terminado”

1- Salida - **Definition of OUTPUT Done:** Define las medidas de calidad necesarias para considerar que un Increment está técnicamente terminado y listo para ser entregado.

- Responde: ¿Construimos correctamente el producto?
- Criterios:
 - Código limpio y testeado
 - Integración correcta
 - Documentación actualizada
 - Cumplimiento de estándares
- Debe cumplirse dentro del Sprint para considerar el incremento

Además:

- No debe debilitarse por presión del tiempo, si falta tiempo se ajusta el alcance, no la calidad
- No es lo mismo que acceptance criteria de una US, estos son específicos del ítem (PBI), el Output Done es un estándar del increment
- Su cumplimiento es responsabilidad colectiva de los developers

2- Resultado - **Definition of OUTCOME Done:** Define la evidencia observable o medible necesaria para demostrar que el Product produjo el beneficio esperado.

- Responde: ¿Construimos el producto correcto? ¿Generamos valor?
- Criterios:
 - Cambio observable en el comportamiento del usuario
 - Impacto medible en el negocio (retorno sobre la inversión realizada ROI)
 - Validación con evidencia de las hipótesis planteadas

Además:

- Conviene definir como se medirá antes de construir para evitar sesgos, el PO tiene la decisión final
- Se prioriza evidencia directa, sobre la circunstancial (métricas internas del scrum team)
- Puede medirse con telemetría del producto (datos reales del uso del producto)
- Suele evaluarse después de liberar el Increment, aunque puede moritorearse antes, durante y despyes de una release

El Output es requisito previo; el Outcome es el éxito real.

Definition of Ready: Indica que un Product Backlog Item tiene claridad suficiente para comenzar a trabajarse. Es una calidad emergente del refinamiento, no un compromiso formal de Scrum.

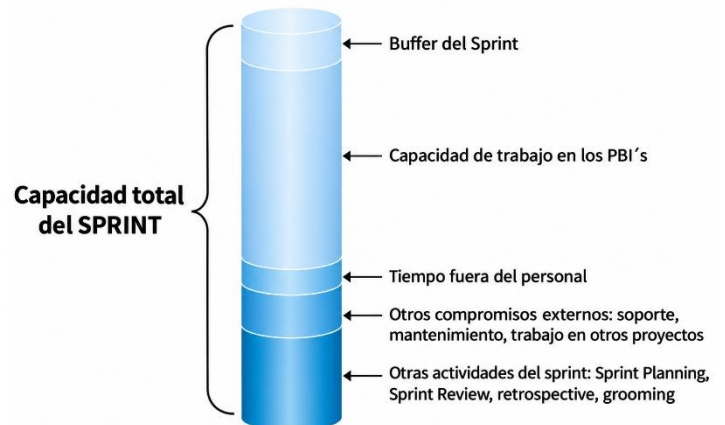
- Claridad suficiente no excesiva, tener suficiente para empezar sin crear desperdicio por sobre-refinamiento
- Un ítem ready incluye:
 - Acceptance criteria (qué debe lograr)
 - Outcome criteria (por qué genera valor)

Capacidad del equipo

La capacidad representa cuánto trabajo puede asumir razonablemente el equipo durante un Sprint. Se utiliza durante el Sprint Planning para decidir cuánto trabajo incorporar.

Capacidad $\neq$ Velocidad

- Capacidad: disponibilidad esperada para un Sprint futuro.
- Velocidad: trabajo efectivamente terminado en iteraciones anteriores.



Ambiente de trabajo

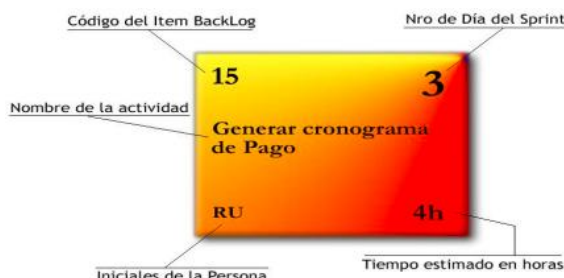
Scrum favorece un ambiente de trabajo que facilite la comunicación, colaboración y autogestión:

- Abierto: debe facilitar la interacción entre los miembros.
- El silencio absoluto es un mal signo: se espera comunicación y colaboración frecuente.
- Pizarrones: ayudan a visualizar y hacer transparente el trabajo.
- El equipo define sus horarios: en línea con la autogestión del Scrum Team.

Herramientas de seguimiento

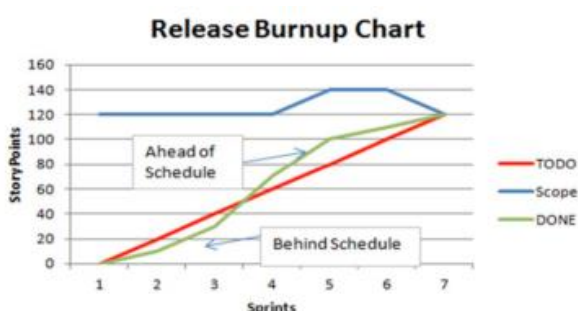
1. **Taskboard:** Representa visualmente el trabajo del Sprint.

Las tareas deberían tener suficiente granularidad para que pueda observarse realmente el avance del Sprint.



Story	To Do	In Process	To Verify	Done
As a user, I... 8 points	Code the... 9	Test the... 8	Code the... DC 4	Test the... SC 6
	Code the... 2	Code the... 8		Test the... SC 6
	Test the... 8	Test the... 4		Test the... SC 6
				Test the... SC 6
As a user, I... 8 points	Code the... 8	Test the... 8	Code the... DC 8	Test the... SC 6
	Code the... 4	Code the... 6		Test the... SC 6

2. **Burn-Down Chart:** Muestra cuánto trabajo queda pendiente a medida que avanza el tiempo. Idealmente presenta una tendencia descendente.



3. **Burn-Up Chart (Grafico de seguimiento del producto):** Muestra cuánto trabajo se ha completado. Presenta una tendencia ascendente.

Niveles de planificación

La planificación ágil se realiza en distintos niveles, desde decisiones de largo alcance hasta la coordinación diaria

A medida que se avanza a niveles más cercanos a la ejecución:

- disminuye el horizonte temporal;
- aumenta el nivel de detalle;
- se reduce la incertidumbre;
- el trabajo se descompone progresivamente.



1. Estrategia: Es el nivel más general. Define hacia dónde quiere ir la organización y orienta qué productos o iniciativas tienen sentido desarrollar.

2. Planificación de Portfolio: Trabaja sobre el conjunto de productos o iniciativas posibles de la organización. El objetivo es decidir en qué iniciativas conviene invertir, priorizando aquellas alineadas con la estrategia y con potencial de generar valor.

3. Planificación de producto: Parte de una idea que supera un filtro estratégico y se transforma progresivamente en una visión del producto.

A este nivel se definen principalmente:

- Product Vision: hacia dónde se pretende llevar el producto
- Product Backlog: qué trabajo podría ser necesario
- Product Roadmap: evolución esperada del producto
- posibles Releases.

no busca definir todo desde el comienzo, sino establecer una dirección que luego se irá ajustando.

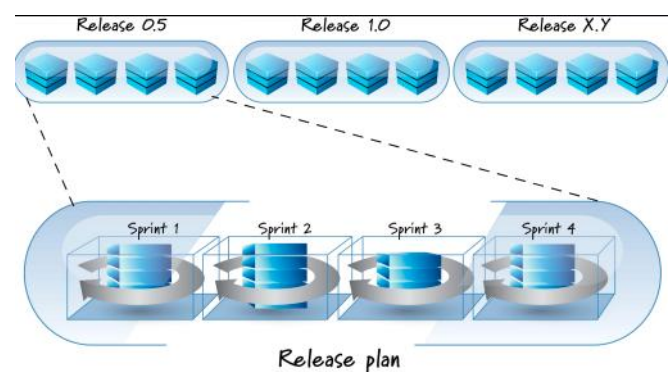
4. Planificación de Release: Determina qué valor se pretende entregar en una Release y cómo distribuirlo a lo largo de varios Sprints. Utiliza como información:

- Lo definido en la planificación de producto (Product Vision; Product Backlog; Product Roadmap)
- velocidad del equipo
- restricciones de fecha, presupuesto y alcance

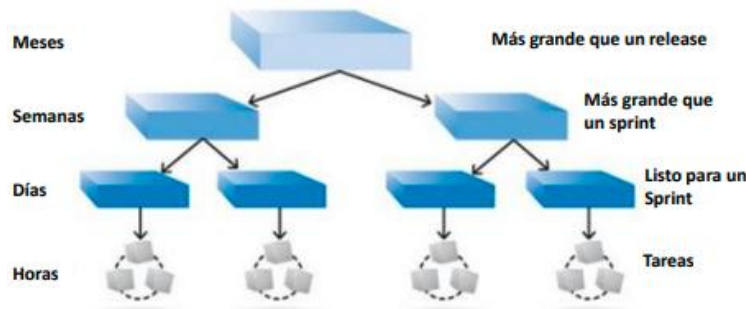
Se seleccionan y ordenan funcionalidades y se las distribuye entre Sprints.

La Velocity puede utilizarse en la planificación de Release para proyectar aproximadamente cuántos Product Backlog Items pueden completarse por Sprint y, por lo tanto, cuántos Sprints puede requerir una Release.

Una Release puede abarcar uno o varios Sprints. El Release Plan permite visualizar cómo se distribuye el trabajo entre ellos.



Granularidad de la planificación: El trabajo se descompone a medida que se acerca su implementación:



A mayor cercanía con la ejecución, mayor detalle y menor tamaño de los elementos planificados.

Esto refleja la idea de planificación progresiva / Just in Time: no se necesita detallar hoy el trabajo que recién se realizará dentro de varios meses.

5. Planificación de Iteración: selecciona qué ítems del Product Backlog trabajar y define cómo realizarlos mediante el Sprint Backlog. Es la primera actividad del Sprint.

6. Planificación de día: Cada día se inspecciona el progreso y se adapta el trabajo durante la ejecución del Sprint.

MÉTRICAS EN AMBIENTES ÁGILES

En Agile se adopta una visión minimalista de las métricas: medir no debe convertirse en una actividad que genere trabajo innecesario.

Regla de oro:

- La medición es una salida, no una actividad.
- Medir lo que sea necesario y nada más.

Las métricas deben aportar información útil para inspeccionar la realidad y tomar decisiones, evitando medir simplemente porque algo puede medirse.

Principios que guían la elección de métricas:

- 1: Satisfacer al cliente mediante entregas tempranas y continuas de software valioso.
- 7: El software funcionando es la principal medida de progreso.

Por eso, las métricas ágiles deberían centrarse principalmente en valor entregado y software realmente funcionando, no sólo en cantidad de tareas realizadas o esfuerzo consumido.

Running Tested Features — RTF: Mide la cantidad de funcionalidades que están funcionando y fueron testeadas a lo largo de las iteraciones. Permite observar el progreso mediante software real y comprobado, en lugar de medir solamente actividades realizadas.

Capacidad: Mide cuanto tiempo hay disponible. Puede variar entre sprints según la disponibilidad real de personas.

Capacidad = Horas de trabajo disponibles por día × Días disponibles en la iteración

Capacidad = WH + DA

- Horas disponibles, refleja la capacidad disponible
- Story Points, sirven para observar la velocidad

Velocity: Observa cuanto trabajo se terminó realmente. Mide story points efectivamente completados por iteración

SOFTWARE CONFIGURATION MANAGMENT - SCM

Software en contexto

Un proceso (adaptado y automatizado con herramientas) se aplica dentro de un proyecto, donde participan personas, y de esa ejecución se obtiene el producto de software



Cuando pensamos en SW pensamos en un conjunto de:

1. Programas
2. Procedimientos
3. Reglas
4. Documentación
5. Datos

El software es información:

- Estructurada con propiedades lógicas y funcionales (no es solo texto suelto, tiene lógica y cumple funciones)
- Creada y mantenida en varias formas y representaciones (documentos, diagramas, código fuente, etc. distintos niveles de abstracción).
- Que, en su forma más desarrollada, está confeccionada para ser procesada por una computadora (el código ejecutable, binario).

Cambios en el software, tienen origen en:

- Cambios en el negocio y nuevos requerimientos
- Soporte de cambios de productos asociados
- Reorganización de las prioridades de la empresa por crecimiento
- Cambios en el presupuesto
- Defectos encontrados a corregir
- Oportunidades de mejora

COMO DISCIPLINA DE SOPORTE:

- Es una actividad paraguas, transversal a todo el proyecto
- Es relevante para el producto en su ciclo de vida (no solo en una etapa)

Disciplinas de soporte del software:

Administración de configuración de software →



HISTORIA

Tiene origen a mediados de 1950. CM "Configuration Managment" se usaba originalmente en desarrollo de hardware y control de producción, luego se adoptó y adaptó para el desarrollo de software

Definición ANSI/IEEE 828, 1990

SCM es una disciplina que aplica dirección y monitoreo administrativo y técnico para:

- Identificar y documentar las características funcionales y técnicas de los ítems de configuración.
- Controlar los cambios de esas características.
- Registrar y reportar los cambios y su estado de implementación.
- Verificar la correspondencia con los requerimientos.

¿POR QUÉ DEBERÍAMOS GESTIONAR LA CONFIGURACIÓN DEL SOFTWARE?

Su propósito es establecer y mantener la integridad de los productos de software a lo largo de su ciclo de vida. Para lograrlo involucra:

- Identificarla en un momento dado (saber que componentes/versiones forman el producto en ese punto)
- Controlar sistemáticamente sus cambios
- Mantener su integridad y origen (trazabilidad: saber de dónde viene cada cambio y que el producto siga siendo consistente)

INTEGRIDAD DEL PRODUCTO

Un producto tiene integridad cuando:

- Satisface las necesidades del usuario
- Puede ser rastreado fácil y completamente durante su ciclo de vida
- Satisface criterios de performance
- Cumple con sus expectativas de costo

El software es un blanco móvil, ya que, está en constante cambio y, por lo tanto, genera:

Problemas en el manejo de componentes

- Pérdida de un componente: Se extravía o no se encuentra.
- Pérdida de cambios: el componente que tengo no es la última versión.
- Sincronía fuente–objeto–ejecutable: Que el código fuente, el compilado y el ejecutable correspondan entre sí (no quedar desactualizados unos respecto de otros).
- Regresión de fallas: Un error ya corregido vuelve a aparecer (por usar una versión vieja o mal sincronizada).
- Doble mantenimiento: Corregir el mismo problema dos veces en distintas copias/versiones porque no hay control centralizado.
- Superposición de cambios: Dos personas modifican lo mismo al mismo tiempo y un cambio pisa al otro.
- Cambios no validados: Se aplican modificaciones sin revisión ni aprobación previa.



CONCEPTOS CLAVE



1. ÍTEM DE CONFIGURACIÓN (IC)

Todos y cada uno de los artefactos que forman parte del producto o proyecto, que:

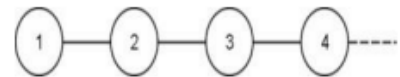
- pueden sufrir cambios
- necesitan ser compartidos entre los miembros del equipo
- sobre los cuales necesitamos conocer su estado y evolución

Elementos de SW que pueden ser guardados en un disco o repositorio.

Ej: Documentos de diseño, Código fuente, Código ejecutable, Foto de un ejercicio resuelto en una hoja, Plan de CM, Requerimientos, Arquitectura del Software, Estándares de codificación, Casos de prueba, Manual de Usuario, Plan de Calidad, Programa de instalación, Registro del proyecto

Versión: Desde el punto de vista de la evolución, es la forma particular de un artefacto en un instante o contexto

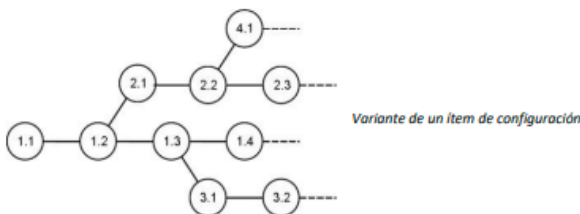
Control de versiones: Evolución de un único ítem de configuración, o de cada uno por separado



Evolución lineal de un ítem de configuración

Variante: Versión de un ítem de configuración (o de la configuración) que evoluciona por separado. Representan configuraciones alternativas.

Un producto de software puede adoptar distintas formas (configuraciones) dependiendo del lugar donde se instale. Por ejemplo, dependiendo de la plataforma (máquina + S.O.) que la soporta, o de las funciones opcionales que haya de realizar o no

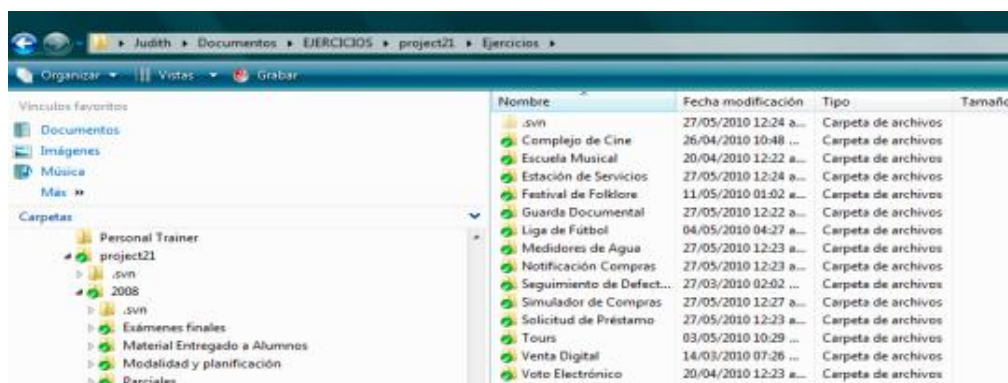


Configuración del software → Un conjunto de ítems de configuración con su correspondiente versión en un momento determinado

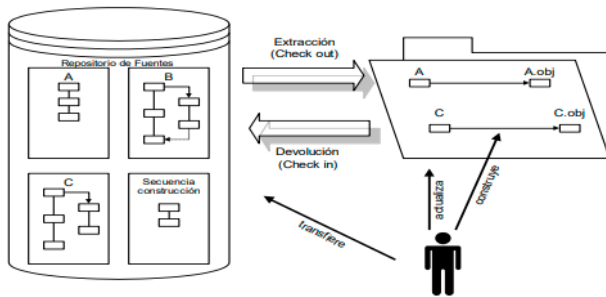
2. REPOSITORIO

Base de información que contiene y organiza los ítems de configuración de un proyecto.

- Mantiene la historia de cada IC, junto con sus atributos y relaciones.
- Se usa para realizar evaluaciones de impacto de los cambios propuestos (analizar qué otros ICs se ven afectados si se modifica uno).
- Puede estar compuesto por una o varias bases de datos.

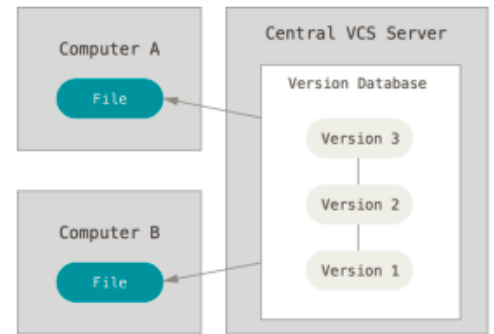


Funcionamiento:

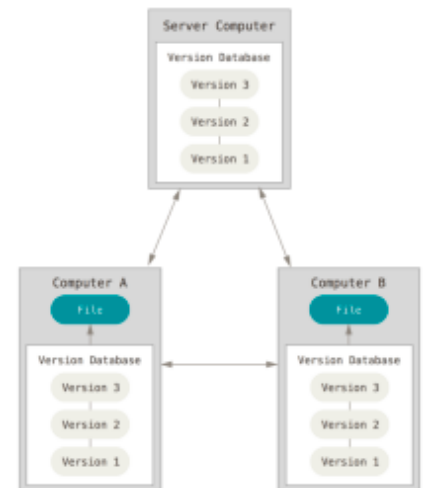


El repositorio centraliza las versiones "oficiales"; el desarrollador saca una copia (check out), trabaja y construye sobre ella, y luego devuelve los cambios (check in) para mantener todo sincronizado y controlado.

Repositorios centralizados → Un servidor contiene todos los archivos con sus versiones. Falla el servidor y estamos "al horno". Los administradores tienen el mayor control sobre este.



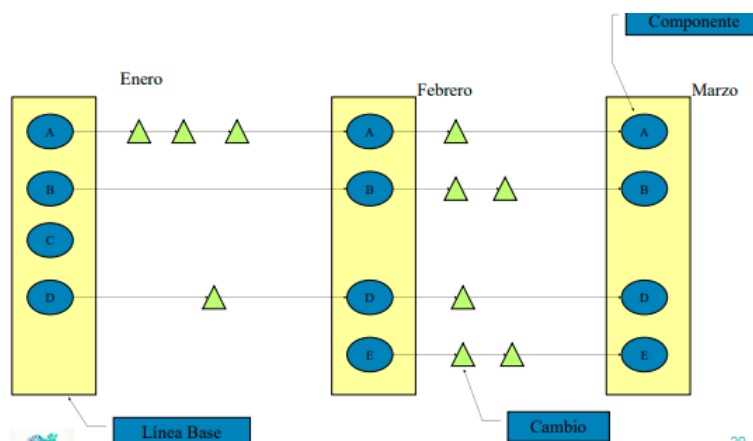
Repositorios descentralizados → Cada cliente tiene una copia exactamente igual del repositorio completo. Si un servidor falla es cuestión de "copiar y pegar". Posibilita otros workflows no disponibles en el modelo centralizado



3. LÍNEA BASE

Es una "foto" formal de una configuración del software en un momento dado, un conjunto de ítems de configuración que ha sido revisado y acordado formalmente, y que a partir de ahí solo puede modificarse mediante un procedimiento formal de control de cambios (no de manera libre).

- Se identifica con etiquetas para "marcar" las baseline (no confundir con el número de versión del producto)
- Sirve como punto de partida estable para desarrollos posteriores.
- Permite volver atrás en el tiempo y reproducir el entorno de desarrollo tal como estaba en ese momento del proyecto.
- No debe confundirse con la versión del producto

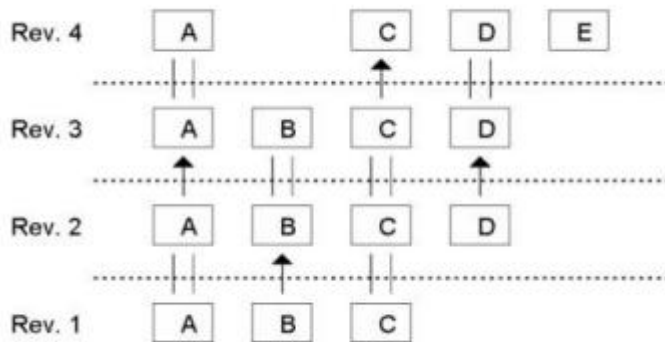


Representación: Pueden ser

- De especificación (Requerimientos, Diseño)
- De productos que han pasado por un control de calidad definido previamente



Evolución de una configuración:



Cada revisión representa el estado de la configuración en un momento, formada por los ítems vigentes.

No todos los ítems cambian en c/revisión: solo se actualizan los que sufrieron una modificación (↑), el resto se mantiene igual.

Pueden agregarse nuevos ítems con el tiempo (D aparece en Rev.2, E en Rev.4).

Cada revisión es una nueva configuración completa, resultado de combinar los ítems que cambiaron con los que permanecieron sin modificar.

4. RAMAS

Es una bifurcación del desarrollo que permite trabajar en paralelo sobre una línea de código independiente, sin afectar la rama principal, hasta que esos cambios estén listos para integrarse (o se descarten).

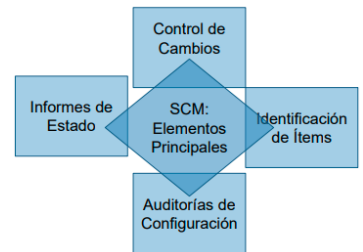
Creación de ramas:

- Existe una rama principal (trunk / master).
- Sirven para bifurcar el desarrollo.
- Pueden tener una razón de creación con semántica.
- Permiten la experimentación sin riesgo sobre la rama principal.
- Pueden terminar de dos formas: integradas o descartadas.

Integración de ramas: La operación se llama **merge**.

- Lleva los cambios de la rama a la rama principal.
- Pueden surgir conflictos, que se resuelven comparando diferencias (diff).
- Regla general: toda rama debería eventualmente integrarse a la principal o ser descartada (no quedar "colgada" indefinidamente).

ACTIVIDADES FUNDAMENTALES DE LA GESTION DE CONFIGURACION DE SW



1- IDENTIFICACION DE ITEMS DE CONFIGURACIÓN (Elemento esencial)

Consiste en determinar qué elementos del proyecto estarán bajo control de SCM y cómo serán identificados y organizados.

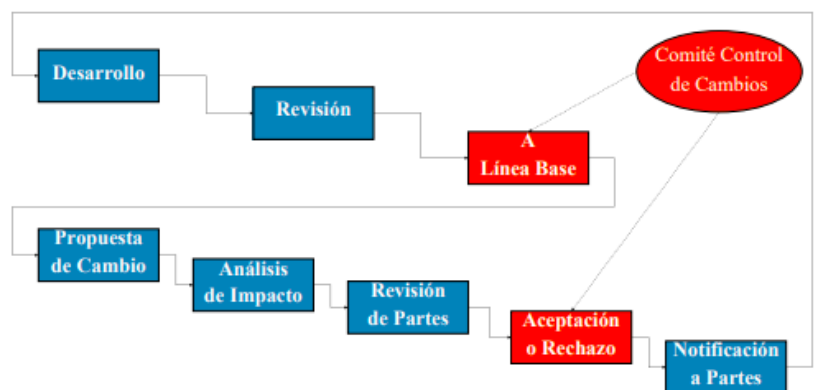
- Identificación unívoca de cada ítem de configuración: Permite distinguirlo de los demás y reconocer su versión, revisión o release.
- Convenciones y reglas de nombrado: Se establecen criterios uniformes para nombrar o numerarlos, facilitando su identificación y control.
- Definición de la Estructura del Repositorio: se determina cómo se organizarán dentro del repositorio, por ejemplo, mediante carpetas.
- Ubicación dentro de la estructura del repositorio: Se define dónde se almacena cada CI dentro de esa estructura para poder localizarlo y recuperarlo correctamente.

CI para un proyecto de desarrollo de software:



2- CONTROL DE CAMBIOS

Procedimiento formal mediante el cual se gestionan modificaciones sobre uno o varios ítems de configuración que ya se encuentran en una línea base. Su objetivo es evitar cambios no controlados y mantener la integridad de la configuración.



1. Propuesta de cambio: surge una necesidad y se documenta formalmente el cambio solicitado.
2. Análisis de impacto: se analiza cómo afectaría el cambio al producto y al proyecto: otros CI, costos, cronograma, pruebas, documentación, etc.
3. Revisión de las partes involucradas: las áreas afectadas evalúan la propuesta y sus consecuencias.
4. Aceptación o rechazo: el Comité de Control de Cambios decide si el cambio se autoriza. Los cambios sobre elementos controlados deben evaluarse antes de ser implementados.
5. Implementación y notificación: si se acepta, se genera la orden de cambio, se incorpora a la cola de cambios y posteriormente se implementa de manera controlada. Si se rechaza, no se modifica la línea base. Las partes interesadas son notificadas de la decisión.

Comité de control de cambios – CCC: Grupo encargado de evaluar y aprobar o rechazar los cambios propuestos sobre los elementos bajo control. Integrado por representantes de las áreas involucradas, análisis, diseño, implementación, testing y otros, de modo que pueda evaluarse el impacto del cambio desde distintas perspectivas.

3- AUDITORIA DE GESTION DE CONFIGURACION

Las auditorías permiten comprobar que los ítems de configuración y las líneas base son correctos y consistentes. Se distinguen dos tipos principales:

Auditoria FÍSICA de configuración: Verifica que el producto construido coincida con su documentación y que lo documentado para cada CI en la línea base o actualización realmente exista y esté implementado.

Auditoria FUNCIONAL de configuración: Verifica que el producto construido cumpla con los requerimientos, tanto en funcionalidad como en desempeño. Una herramienta fundamental es la matriz de trazabilidad de requisitos, que permite seguir cada requerimiento y determinar, por ejemplo, qué archivos de código lo implementan y qué pruebas se realizaron para verificarlo.

Herramienta básica para ambas: checklist

Sirve como apoyo para estos 2 procesos básicos:

- **Validación:** comprueba que el producto resuelva adecuadamente el problema y satisfaga las necesidades del usuario. ¿Construimos el producto correcto?
- **Verificación:** comprueba que el producto cumpla con lo especificado en la documentación y líneas base. ¿Construimos el producto correctamente?

4- INFORMES DE ESTADO

Se ocupa de mantener los registros de la evolución del sistema.

Permite mantener la trazabilidad de los cambios, registrando qué cambió, cuándo, por qué, quién lo autorizó y en qué versión se incorporó. Como maneja una gran cantidad de información, suele implementarse mediante herramientas o procesos automáticos.

Incluye reportes de rastreabilidad de todos los cambios realizados a las líneas base durante el ciclo de vida

Gracias a estos registros se pueden responder preguntas como:

- ¿Cuál es el estado actual de un CI?
- ¿Un cambio fue aprobado o rechazado por el CCB/CCC?
- ¿Qué versión de ítem implementa un requerimiento de cambio aprobado?
(saber cuál es el componente que contiene la mejora)
- ¿Qué diferencias existen entre dos versiones?

PLAN DE GESTION DE CONFIGURACION

El Plan de Gestión de Configuración define cómo se aplicará SCM en el proyecto, estableciendo las reglas, responsabilidades, herramientas y procedimientos que se utilizarán.

Debe incluir principalmente:

1. Reglas de nombrado de los CI
2. Herramientas a utilizar para SCM
3. Roles e integrantes del Comité
4. Procedimiento formal de cambios
5. Plantillas de formularios
6. Procesos de Auditoría

GESTION DE CONFIGURACION DE SW en AMBIENTES ÁGILES

En metodologías ágiles, SCM sigue siendo necesario, pero se adapta para acompañar al equipo y facilitar los cambios en lugar de introducir controles pesados.

- Sirve al equipo de desarrollo, busca facilitar el trabajo de los practicantes, no agregar burocracia.
- Coordina en lugar de controlar, permite hacer seguimiento del desarrollo sin imponer un control rígido sobre los desarrolladores.
- Responde al cambio, se considera algo esperado y se gestiona de manera rápida, en lugar de intentar evitarlo.
- Esforzarse por ser transparente y generar la menor “fricción” posible, automatizando tareas repetitivas.
- Coordinación frecuente y rápida: las actividades de configuración se realizan continuamente, acompañando las iteraciones cortas.
- Eliminar desperdicio, se evita documentación o tareas de SCM que no aporten valor.
- Documentación Lean y trazabilidad, se mantiene solo la documentación necesaria, pero sin perder la posibilidad de rastrear cambios.
- Feedback continuo, se busca tener información visible y frecuente sobre la calidad, estabilidad e integridad del producto.

Además, SCM es responsabilidad de todo el equipo, no exclusivamente de una persona o área. Sus actividades se integran dentro de las tareas necesarias para alcanzar el objetivo del Sprint, automatizando lo máximo posible.

¿Qué cambia respecto del SCM tradicional?

Las actividades fundamentales no desaparecen, sino que se adaptan al enfoque ágil:

- Comité de Control de Cambios: se reduce la necesidad de un comité formal para cada cambio. La gestión de cambios se integra al trabajo cotidiano del equipo.
- Ítems de configuración: siguen existiendo; pueden ser código, pruebas, documentación, historias, configuraciones, etc., priorizando aquellos que realmente aportan valor controlar.
- Auditorías: no necesariamente se realizan como una gran actividad formal al final; los controles pueden integrarse y automatizarse durante el desarrollo.
- Reportes de estado: se busca que sean continuos y visibles, obtenidos en lo posible automáticamente de las herramientas utilizadas por el equipo.